



Tema 18.3: Agentes y Vibe Coding

Contenidos

- Agentes de IA
- Arquitectura de un Agente
- Demo n8n/OpenAI Agent Builder
- Ejemplos Agentes
- ¿Qué es Vibe Coding?
- Herramientas Low-vibe Coding
- Beneficios, Riesgos y Seguridad
- Herramientas de Coding Agents
- Buenas prácticas
- MCP: Model Context Protocol
- Spec-Driven Development
- Ejercicios Opcionales



Agentes de IA



AGENTES DE IA



¿Qué pasaría si la IA no solo respondiera preguntas...

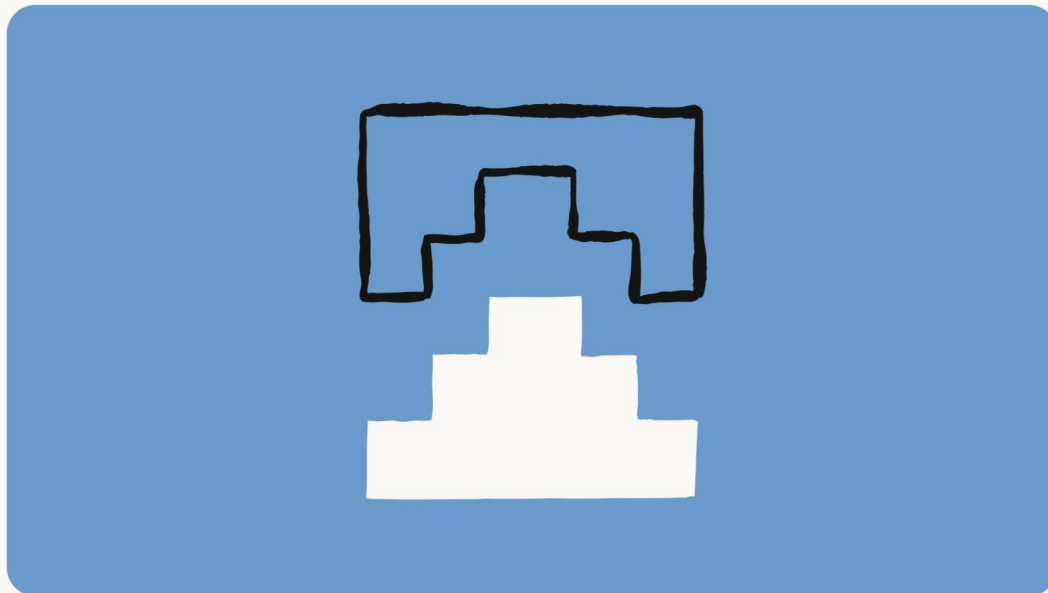
...sino que tomara **iniciativas**, planificara **estrategias** y ejecutara **tareas** de forma **autónoma**?



Announcements

Anthropic acquires Bun as Claude Code reaches \$1B milestone

3 dic 2025



Claude is the world's smartest and most capable AI model for developers, startups, and enterprises. Claude Code represents a new era of agentic coding, fundamentally changing how teams build software. In November, Claude Code achieved a significant milestone: just six months after becoming available to the public, it reached \$1 billion in run-rate revenue. And today we're announcing that Anthropic is acquiring [Bun](#)—a breakthrough JavaScript runtime—to further accelerate Claude Code.



Manus ahora forma parte de Meta →

Producto · miércoles, diciembre 17

Actualización de Manus: 100 M\$ en ARR, 125 M\$ en tasa de ingresos



Manus ha superado los 100 millones de dólares en ARR, ocho meses después de su lanzamiento. Esto nos convierte en la startup que más rápido ha pasado de 0 a 100 millones de dólares en el mundo. La tasa de ingresos totales de nuestra empresa supera ahora los 125 millones de dólares, incluyendo los ingresos basados en el uso y otros. Hemos estado creciendo más de un 20 % mes a mes desde el [lanzamiento de Manus 1.5](#), y seguimos acelerando el lanzamiento de nuestros productos y funciones.

Evolución: Del Chatbot al Agente Autónomo

2020-2022

Modelos Reactivos

GPT-3, BERT

Generación de texto

Sin memoria

Respuestas aisladas

Responden

2023

Razonamiento

Chain-of-thought

Análisis paso a
paso Documentos
largos

Coherencia
lógica

Razonan

2024

Herramientas

GPT-4 + Plugins

Acceso a
Internet

Integración APIS
Multi-paso

Actúan

2025

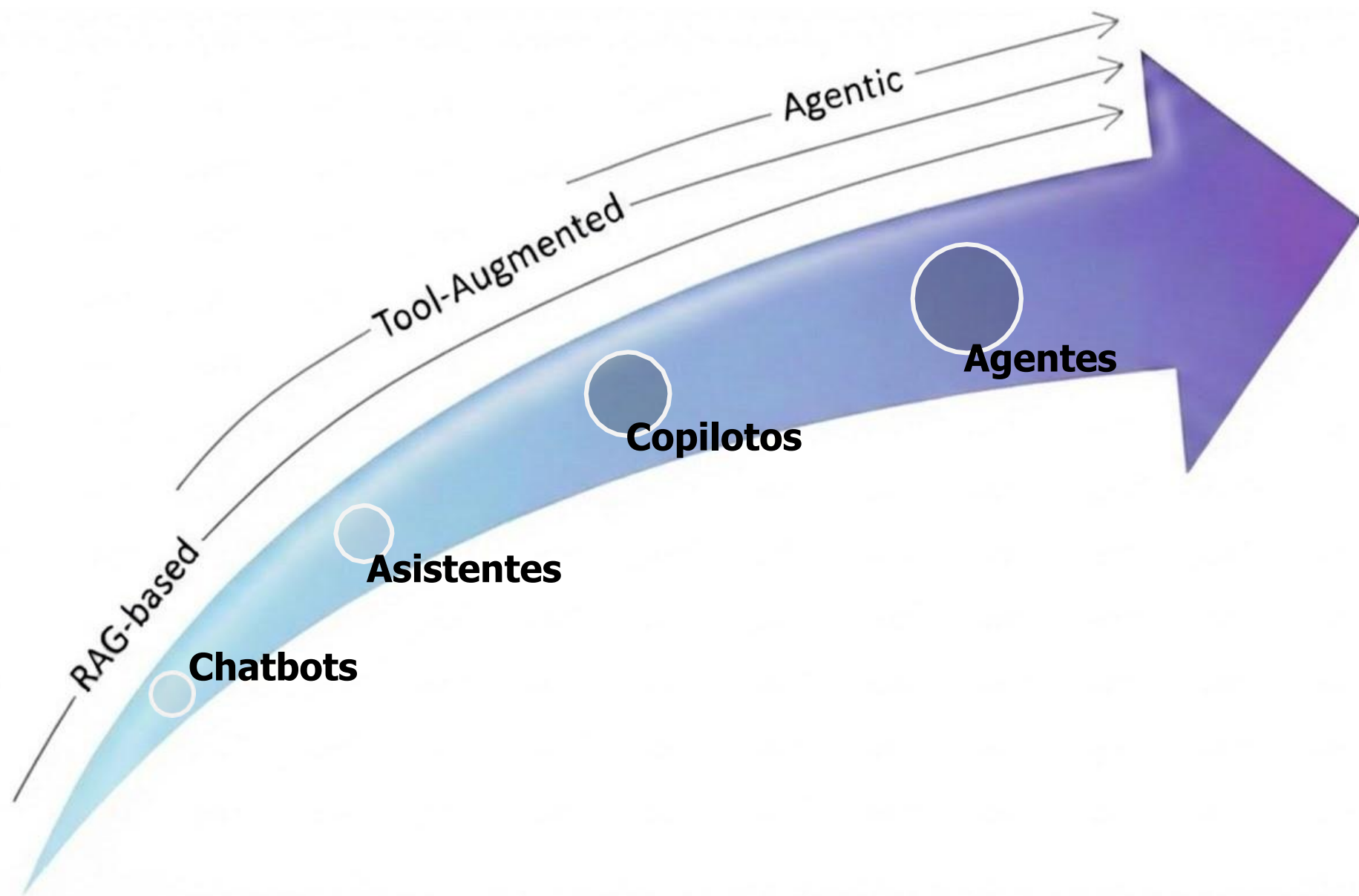
Agentes Autónomos

Memoria persistente

Aprendizaje continuo
Autonomía supervisada

Auto-mejora

Deciden



Arquitectura de un agente



Definición: ¿Qué es un Agente de IA?



Percibir

Observan su entorno mediante herramientas, lectura de archivos o inputs de usuario.



Decidir

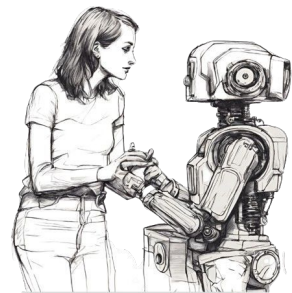
Razonan sobre la información para planificar la mejor secuencia de acciones.



Ejecutar

Actúan sobre el mundo real invocando APIs, escribiendo código o enviando mensajes.

En resumen: Son programas **autónomos** que persiguen un **objetivo interactuando** con **herramientas** y **razonando** sobre los resultados.



Agentes Autónomos

Operan y toman decisiones por sí mismos **sin intervención humana.**

Agentes Cooperativos

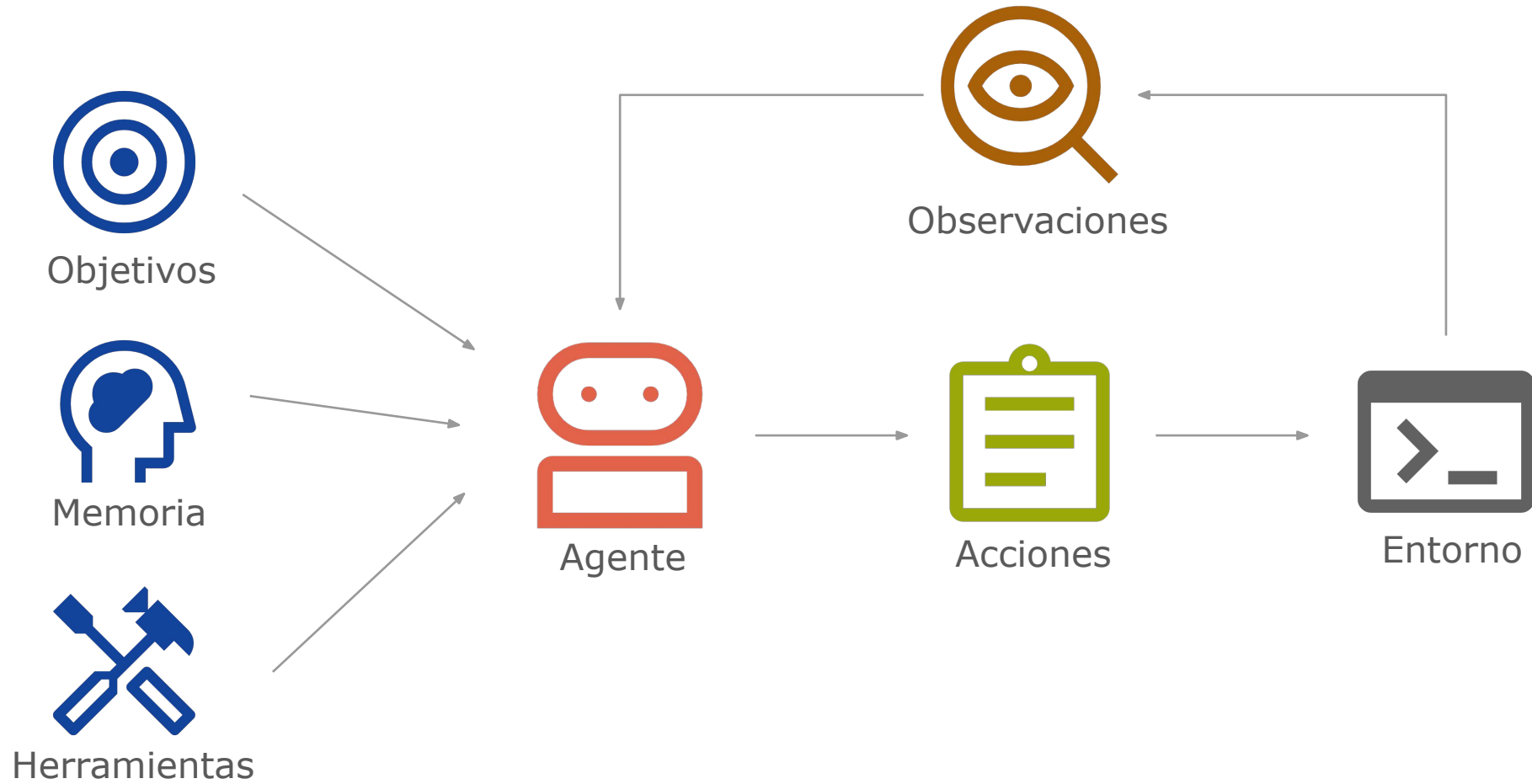
Trabajan juntos para resolver un problema.

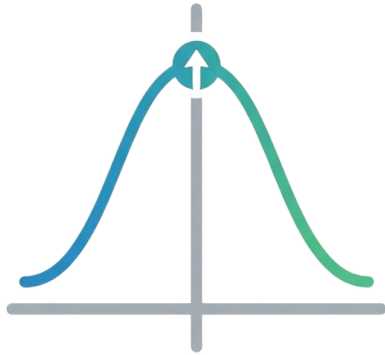
Agentes de Aprendizaje

Aprenden de las **experiencias pasadas** y mejoran su rendimiento.

Multi-Agentes

Un sistema donde **múltiples agentes** interactúan entre sí para lograr un objetivo.





Objetivos Simples

Encuentra el mejor precio para X



Objetivos Complejos

Optimiza la cadena de suministro



Objetivos Emergentes

Mejora continuamente el proceso





Memoria de trabajo (Contexto)

- Tarea actual y conversación activa
- Archivos abiertos / estado temporal
- Se pierde entre sesiones y está limitada por tokens
- Dónde vive hoy: Prompt, chat, ventana de contexto



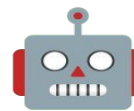
Memoria episódica

- Experiencias pasadas del agente
- Decisiones tomadas y errores aprendidos
- Historial de sesiones o incidencias
- Dónde vive hoy: MEMORY.md, logs, notas de sesión



Memoria semántica (RAG)

- Conocimiento estable del proyecto
- Conceptos, arquitectura y reglas de negocio
- Documentación recuperable bajo demanda
- Dónde vive hoy: Wiki, docs, RAG, AGENTS.md



Memoria procedimental

- Cómo hacer tareas paso a paso
- Workflows, checklists y estándares
- Scripts, comandos y plantillas reutilizables
- Dónde vive hoy: SKILL.md, runbooks, scripts/resources

Context Engineering

¿Dónde se materializa esa memoria hoy?

AGENTS.md

- Reglas comunes del proyecto para distintos agentes
- Suele contener comandos, estándares y contexto compartido

CLAUDE.md

- Instrucciones específicas para Claude Code
- Añade preferencias, imports y contexto propio de Claude

SKILL.md

- Procedimientos reutilizables para tareas concretas
- Puede incluir scripts, recursos, plantillas y ejemplos

MEMORY.md

- Notas o aprendizajes acumulados durante sesiones
- Representa memoria episódica o auto-generada

Archivos específicos de herramienta

- cursorrules, copilot-instructions.md, GEMINI.md, JULES.md...
- Adaptan las mismas reglas a cada ecosistema

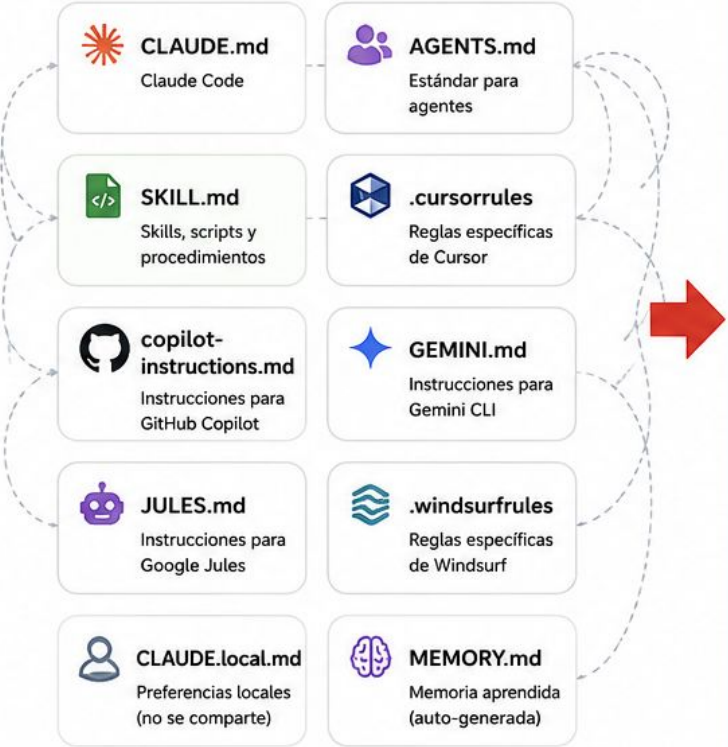
¿dónde guardamos cada tipo de memoria para que la entiendan todos los agentes?

EL PROBLEMA DE LOS .MD PARA AGENTES

Cada herramienta define su propia “memoria”, pero muchas veces contienen las mismas reglas e información.

CAOS ACTUAL

Demasiados archivos .md distintos para lo mismo.



EL PROBLEMA



HACIA LA HOMOGENEIZACIÓN

Un modelo en capas: lo común en común, lo específico donde aporta valor.



OBJETIVO

Reducir la fragmentación, mantener la información actualizada y hacer que cualquier agente entienda tu proyecto.



BENEFICIO

Menos duplicación, reglas consistentes y agentes más efectivos desde el primer momento en cualquier herramienta.

Compile

- Vuelcas docs, papers o repos en /raw.
- El LLM extrae ideas, crea páginas .md y enlaza conceptos.
- Knowledge graph en lenguaje natural.

Query

- El LLM lee la wiki ya sintetizada — no los chunks crudos.
- Las respuestas vuelven a la wiki como páginas nuevas.
- El conocimiento se acumula.

Lint

- El LLM audita: contradicciones, páginas huérfanas, conceptos sin página propia.
- Mantenimiento que ningún humano haría.

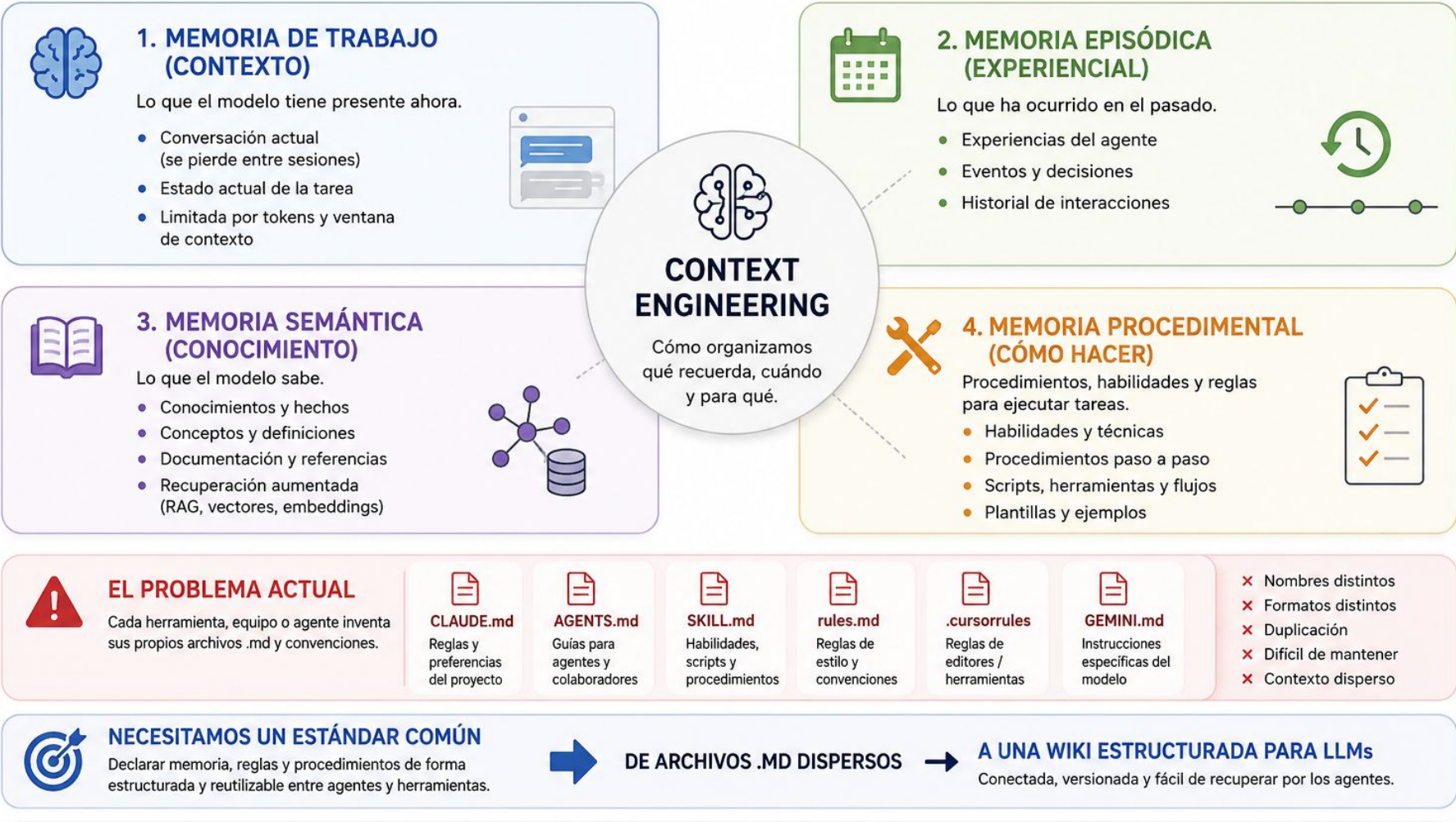
Bye bye RAG: en vez de redescubrir chunks crudos en cada query, la wiki pre-compila y acumula.
Stack: Obsidian (vault markdown) + agente vía MCP.

Karpathy, abril 2026.

<https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f>

TIPOS DE MEMORIA EN LLMs Y ARCHIVOS .MD

Visión global de cómo los agentes recuerdan, aprenden y ejecutan





Análisis

Calculadora, Estadísticas,
Visualización



Web

Search, Scraping,
APIs



Data

SQL, noSQL, File Parsing



AI/ML

Vision, Audio, Video, Otros
agentes



Código

Ejecución, Debugging,
Testing



Sistema

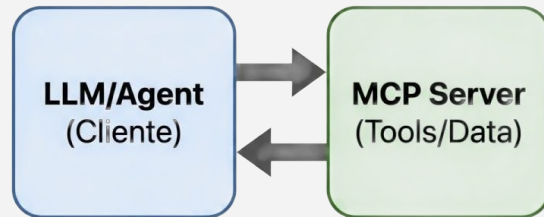
Shell, Ficheros, Procesos

MCP / A2A / Skills / CLIs

MCP

Anthropic - Nov 2024

Propósito: Conectar LLMs con herramientas externas



Características:

- Open-source, modelo-agnóstico
- Adoptado por OpenAI, Google
- "USB-C para AI"
- JSON-RPC 2.0

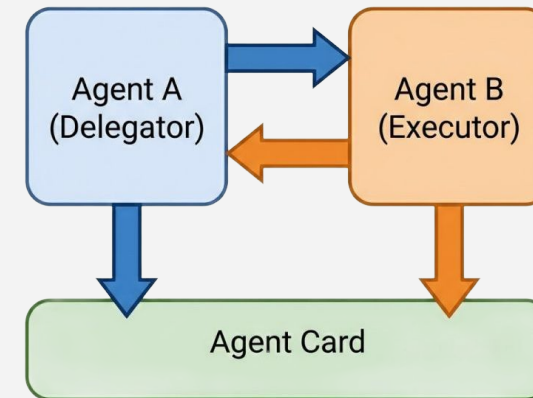
Servidores pre-construidos:

Google Drive, Slack, GitHub, Postgres

A2A (Agent-to-Agent Protocol)

Google - Abril 2025

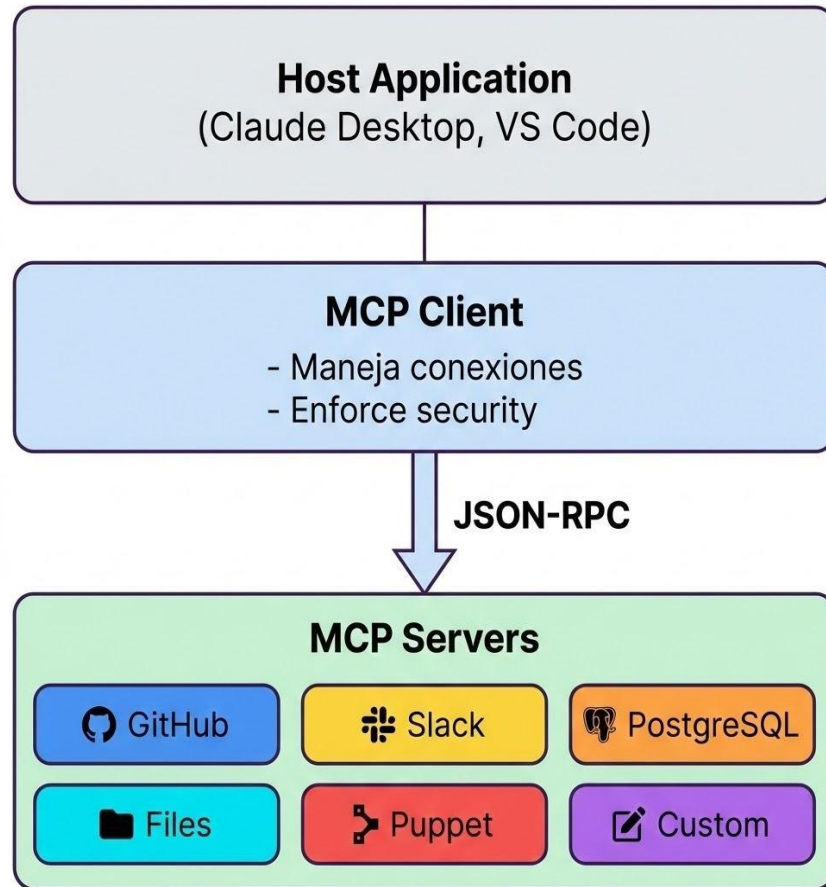
Propósito: Comunicación entre agentes



Características:

- 50+ partners (Salesforce, SAP, etc.)
- Agent Cards (capacidades en JSON)
- HTTP + JSON-RPC
- Linux Foundation

Arquitectura



Configuración

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "..."
      }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "..."
      }
    }
  }
}
```

Un servidor puede atender a muchos clientes. Un cliente puede conectarse a muchos servidores. Escríbelo una vez, úsalo en todas partes.

Servidor MCP

Declara capacidades mediante un esquema JSON:

Herramientas

Funciones invocables por el agente (query_db, search_docs, create_ticket)

Recursos

Datos que el agente puede leer (archivos, filas BD, respuestas API)

Prompts

Plantillas de prompt reutilizables para tareas comunes

Transporte: stdio o HTTP+SSE

Protocolo

Mensajes JSON-RPC 2.0:

1. initialize

Cliente → Servidor:
handshake y capacidades

2. tools/list

Descubre herramientas disponibles

3. tools/call

Invoca una herramienta; el servidor devuelve resultado

4. resources/read

Lee un recurso por URI

Sesión con estado y bidireccional.

Cliente MCP

El agente de IA:

Descubre herramientas al iniciar mediante tools/list

Decide qué herramienta llamar según la intención del usuario

Envía tools/call con argumentos

Recibe resultados estructurados y los incorpora al contexto

MCP Inspector v0.21.2

Transport Type

Streamable HTTP

URL

https://mcp.deepwiki.com/mcp

Connection Type

Direct

Server Entry

Servers File

> Authentication

> Configuration

Reconnect

Disconnect

Connected

DeepWiki

Version: 2.14.3

Resources

Prompts

Tools

Tasks

Apps

Ping

Sampling

Elicitations

Roots

Auth

Metadata

Tools

List Tools

Clear

read_wiki_structure

Get a list of documentation topics for a GitHub repository. Args: repoName: GitHub repository in owner/repo format (e.g....

read_wiki_contents

View documentation about a GitHub repository. Args: repoName: GitHub repository in owner/repo format (e.g. "facebook/react")

ask_question

Ask any question about a GitHub repository and get an AI-powered, context-grounded response. Args: repoName: GitHub repository ...

read_wiki_structure

Get a list of documentation topics for a GitHub repository.

Args: repoName: GitHub repository in owner/repo format (e.g. "facebook/react")

Read-only

Destructive

Idempotent

Open-world

repoName *

facebook/react

Tool-specific Metadata:

Add Pair

No metadata pairs.

Output Schema:

Expand

```
{  "0": "result"  }  x-fastmcp-wrap-result: true}
```

History

Clear

8. tools/call

7. tools/list

6. tools/list

5. tools/list

4. prompts/list

Server Notifications

Clear

No notifications yet

Light

Checkout

Sesiones unificadas: carrito, pricing dinámico e impuestos.

Ej: cart-to-pay sin re-introducir datos en cada retailer.

Identity Linking

OAuth 2.0: vincula usuario ↔ retailer sin compartir credenciales.

Ej: login agéntico autorizado y revocable.

Order Management

Webhooks en tiempo real: estado, tracking, returns.

Ej: notificación push de envío al agente.

Lenguaje común agentes ↔ comercio. Co-desarrollado por Google, Shopify, Etsy, Walmart, Target, Wayfair.
Complementa MCP (tools) + A2A (agent comms) + AP2 (payments).

+ Nueva conversación

Borrar todas

CONVERSACIONES

organizar lista de la compra

4 May · 18:34

opciones de compra de café ...

4 May · 18:30

Nueva conversación

4 May · 14:41

comprar café recomendacio...

4 May · 14:40



ayudame a hacer la compra



5 productos



2 en varios merchants



mejor combinación 23.19 EUR

MERCHANT



Mercado Norte

3 productos



OLIVAR NORTE

también en 1 más

Aceite de oliva virgen extra Arbequina 1 L

GTIN 8410076812345

SKU M1-AOVE-1L

8.95 EUR

más barato



VALLEVERDE

también en 1 más

Leche semidesnatada UHT 6 x 1 L

GTIN 8480000401892

SKU M1-LECHE-6X1

6.49 EUR

más barato



CAMPO CLARO

Arroz redondo extra 1 kg

GTIN 8410184010007

SKU M1-ARROZ-1KG

2.10 EUR

Quedan 180 uds

MERCHANT



Mercado Sur

2 productos



PASTA ITALIANA

Spaghetti nº 5 500 g

GTIN 8076809513388

SKU M2-PASTA-500

1.45 EUR

Quedan 200 uds



MAR ATLÁNTICO

Atún claro en aceite de oliva pack 3x80g

GTIN 8410020112345

SKU M2-ATUN-PACK

4.20 EUR

Quedan 95 uds

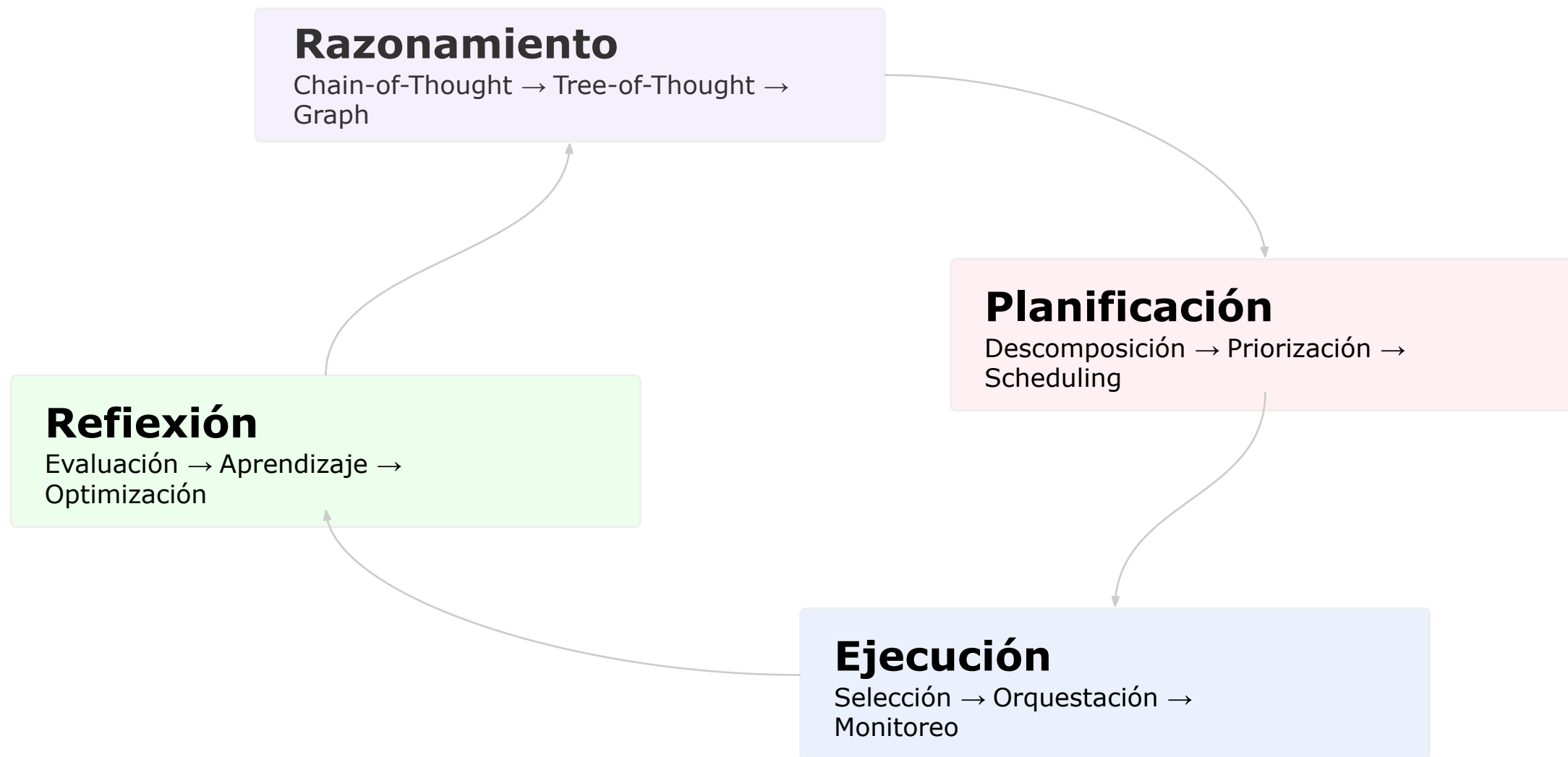
► search_multiple_products{"queries": ["leche 1 litro", "pan 1 barra", "huevos 12 unidades", "arroz 1 kg", "pasta 500 g", "tomates 1 kg", "pollo 1 kg", "aceite de oli..."]}

He leído tu foto: una lista de la compra con 10 items:



Pídeme que busque o compre algo...

Enviar



Chain-of-Thought (CoT)

Razonamiento paso a paso lineal.

Ideal para:
problemas
secuenciales

Tree-of-Thought (ToT)

Exploración de múltiples caminos

Ideal para: problemas
con backtracking

ReAct (Reasoning + Acting)

Intercala pensamiento con acción

Ideal para:
tareas
interactivas

Reflexión

Auto-evaluación y mejora iterativa

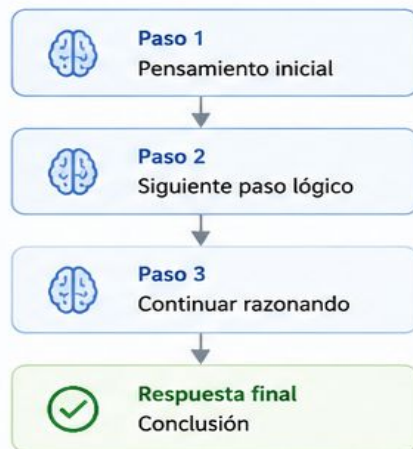
Ideal para:
optimización
continua

ARQUITECTURAS DE RAZONAMIENTO

Distintas formas de organizar el pensamiento, la acción y la mejora

1. CHAIN-OF-THOUGHT (CoT)

Razonamiento paso a paso lineal y secuencial.



🎯 Ideal para:

- Problemas secuenciales
- Tareas con pasos claros
- Explicaciones y derivaciones
- Cálculos y lógica directa

Mentalidad: Profundizar en una sola línea de pensamiento.

2. TREE-OF-THOUGHT (ToT)

Exploración de múltiples caminos y selección del mejor.



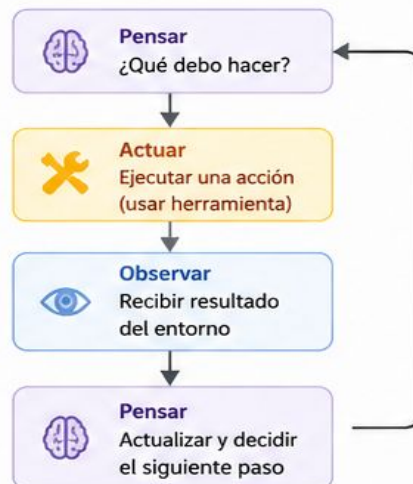
🎯 Ideal para:

- Problemas complejos con alternativas
- Decisiones y planificación
- Backtracking y poda de ramas
- Encontrar la mejor solución

Mentalidad: Explorar muchas opciones antes de decidir.

3. REACT (REASONING + ACTING)

Intercala pensamiento con acción usando herramientas o entornos.



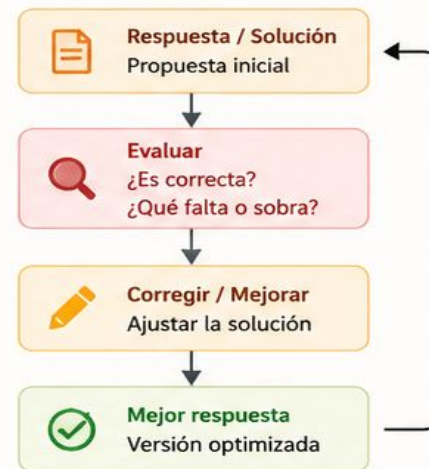
🎯 Ideal para:

- Tareas interactivas
- Uso de herramientas / APIs
- Agentes autónomos
- Entornos dinámicos

Mentalidad: Pensar → Actuar → Observar → Aprender → Repetir.

4. REFLEXIÓN

Auto-evaluación y mejora iterativa del resultado.*



🎯 Ideal para:

- Optimización continua
- Mejora de calidad
- Revisión y corrección
- Aprendizaje de errores

Mentalidad: Generar → Evaluar → Mejorar → Repetir hasta optimizar.



NO HAY UNA ÚNICA MEJOR ARQUITECTURA

Elige la estrategia adecuada según el problema, el contexto, los objetivos y los recursos disponibles.



SE PUEDEN COMBINAR

Estas arquitecturas no son excluyentes.

Pueden usarse juntas para resolver problemas aún más complejos.

SYSTEM PROMPT

Persona: Eres un experto en...

Contexto: Trabajamos sobre el repo X.

Herramientas: Tienes acceso a ls, grep...

Restricciones: Nunca borres archivos.

Formato: Responde siempre en JSON.



Persona y Rol

Define el tono y el dominio de conocimiento.



Tool Definition

Cómo y cuándo usar las herramientas disponibles.



Guardrails

Límites éticos, técnicos y de seguridad.

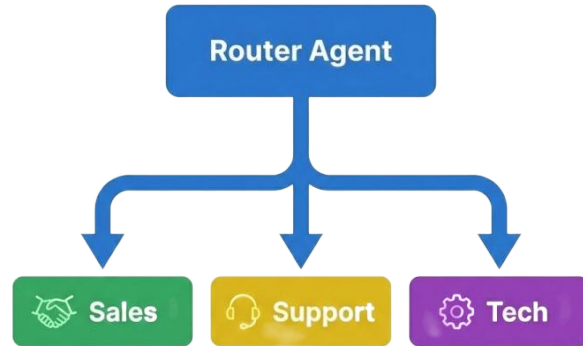


Output Format

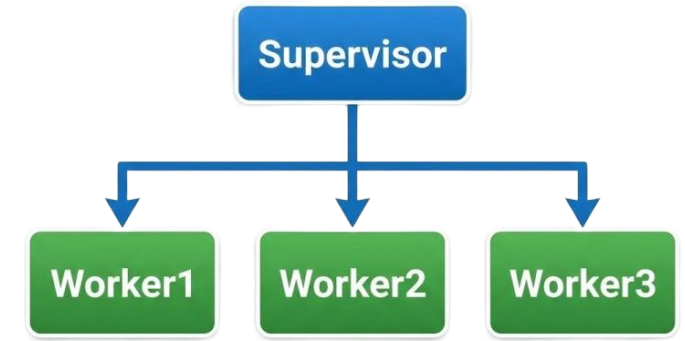
Estructura para que otros sistemas procesen la salida.

Patrones de diseño multi-agente

Router



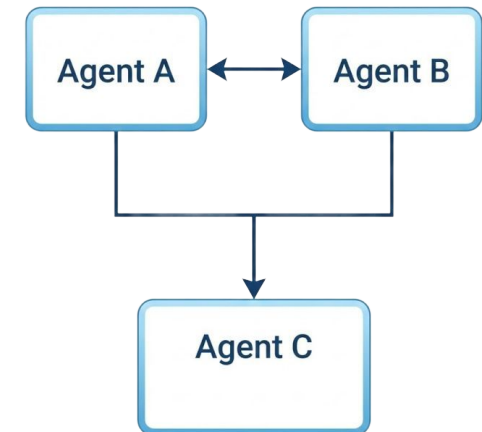
Jerarquía



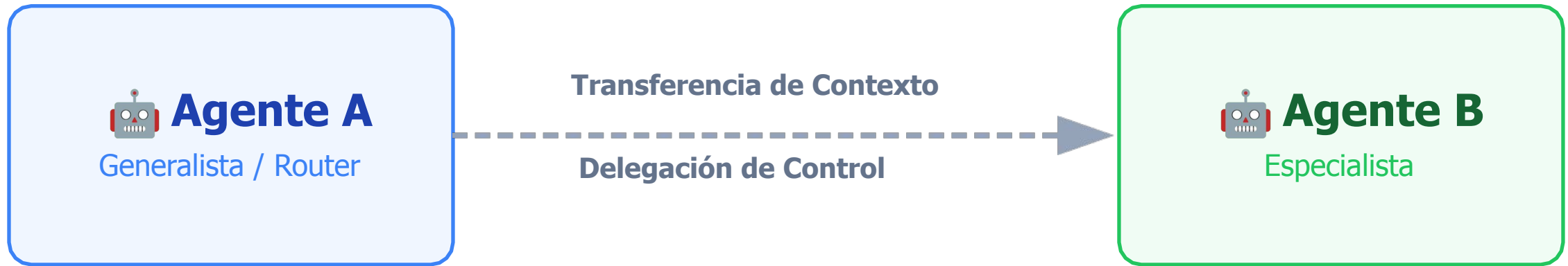
Pipeline



Colaboración



Handoffs: Delegación entre agentes



1. El Agente A agota sus herramientas o detecta una sub-tarea específica.
2. Se invoca al Agente B pasando el historial relevante.
3. El Agente B puede devolver el control o finalizar la tarea.

Comunicación

Protocolos compartidos (JSON, Markdown) y Handoffs explícitos entre agentes.

Estado compartido

Un 'pizarrón' (Blackboard) donde los agentes leen y escriben el progreso.

Memoria Multi-Nivel

- Corto plazo: Ventana de contexto actual (hilos de conversación).
- Largo plazo: Bases de datos vectoriales (RAG) o logs de ejecuciones pasadas.

Human-in-the-Loop

¿Qué es?

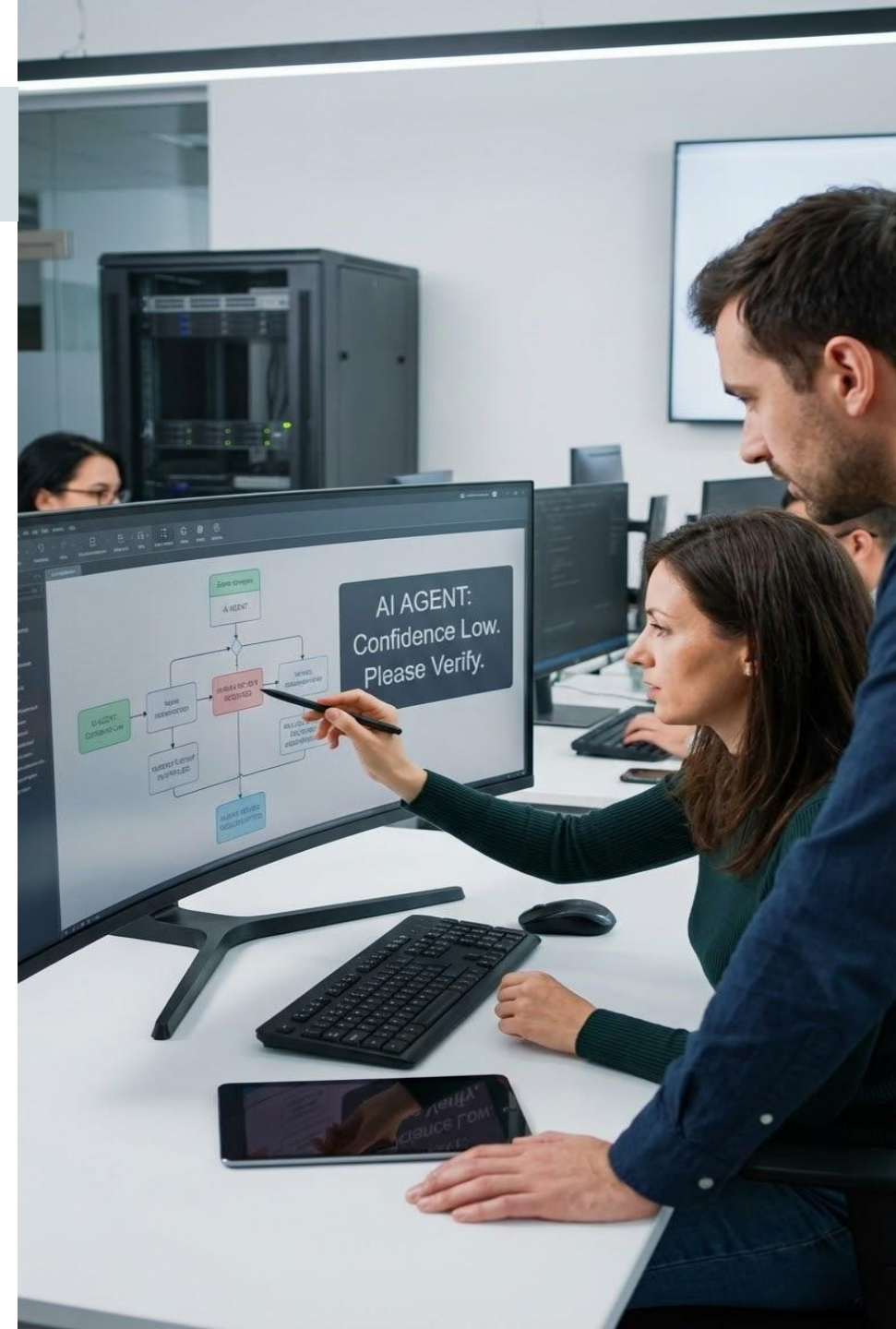
Un humano supervisa y aprueba las decisiones críticas del agente.

Niveles de Autonomía

- Nivel 1: Agente sugiere → Humano ejecuta
- Nivel 2: Agente ejecuta → Humano revisa
- Nivel 3: Agente autónomo → Humano audita
- Nivel 4: Agente totalmente autónomo

Recomendación

Empezar en Nivel 1-2 y escalar gradualmente según confianza.



 <https://mcp.deepwiki.com/mcp>

Lista de herramientas actualizada

✕

<https://mcp.deepwiki.com/mcp>

Permisos de herramientas

Elige cuándo Claude puede usar estas herramientas.

▼ Otras herramientas 3

... Personalizado ▼

read_wiki_structure

✓

👤

✗

read_wiki_contents

✓

👤

✗

ask_question

✓

👤

✗



Compras > X €

Cualquier transacción que supere el umbral aprobado en presupuesto.

Ej.: pedidos de compra, contratación de SaaS, viajes.



Datos sensibles (PII)

Acceso, exportación o cruce de datos personales o regulados.

Ej.: clientes, empleados, salud, finanzas.



Comunicación externa

Mensajes que salen de la organización (email, redes, soporte).

Ej.: respuestas a clientes, prensa, partners.



Acciones irreversibles

Operaciones que no se pueden deshacer fácilmente.

Ej.: borrar registros, despliegue prod, pagos.



Success rate

% de tareas completadas correctamente sin intervención humana.

Objetivo: > 85% en producción.



Latencia

Tiempo medio extremo a extremo por tarea (p50 y p95).

Clave para UX y para tareas en cadena.



Coste por tarea

Tokens × precio + APIs externas + infra.

Vigilar regresiones al cambiar de modelo o prompt.



Intervenciones humanas

% de ejecuciones que requieren rescate o aprobación.

Debe bajar con el tiempo.

Y siempre: satisfacción del usuario y revisión cualitativa de las salidas.



Autonomía y responsabilidad

Guardarraíles y human in the loop



Sesgos

*Evaluación y
testing*



Alucinaciones y confiabilidad

Grounding y verificación cruzada



Latencia y costo

*Caché, compactación y
SLM*



Explicabilidad y transparencia

*Logging, interpretabilidad y
Auditorías*



Cultura

*Proyectos
transversales*



n8n

- **Qué es:** plataforma open-source de workflows automatizables con nodos visuales y nodos AI nativos.
- **Fortaleza:** control total, self-hosted, +500 integraciones, lógica condicional explícita.
- **Ideal para:** equipos que ya hacen automatización, integración con sistemas internos, on-prem.
- **Coste / curva:** cloud 20-50 €/mes o self-hosted gratis. Curva media (entiende JSON, branching).



OpenAI Agent Builder

- **Qué es:** builder visual de OpenAI para crear agentes con tools, handoffs y ChatKit, sobre el Agents SDK.
- **Fortaleza:** prototipado rapidísimo, modelos OpenAI integrados, deploy directo como chat o API.
- **Ideal para:** equipos con foco en LLM, casos conversacionales, demos a negocio.
- **Coste / curva:** pay-as-you-go (tokens). Curva muy baja, sin infra propia.

APIs y HTTP Requests



¿Que es una API?

Antes de hablar de agentes que usan herramientas externas, necesitamos entender como se comunican los sistemas.

Restaurante

La aplicacion o servicio al que quieres acceder.
Ej: OpenAI, Google Maps, Spotify.

Mesa / Cliente

Tu aplicacion o agente.
El que hace la peticion y espera la respuesta.

Camarero

La API.
El intermediario que lleva tu peticion a la cocina y te trae el resultado.

Cocina

El servidor interno.
Procesa la peticion y devuelve los datos. Tu no lo ves.

HTTP Request = como se entrega el pedido

La API es QUE puedes pedir. El HTTP Request es COMO se manda esa peticion por internet.

GET

Pedir datos
Leer informacion

POST

Enviar datos
Crear algo nuevo

PUT

Actualizar
Modificar un recurso

DELETE

Borrar
Eliminar un recurso

En n8n cada nodo que conecta con un servicio externo hace una llamada API por debajo. No necesitas escribirlo — n8n lo gestiona por ti.

La API en la practica — ejemplo real con OpenAI

Cuando tu agente llama a GPT-4o, por debajo esta haciendo esto:

```
# Lo que tu agente envia (HTTP POST)
POST https://api.openai.com/v1/chat/completions
Authorization: Bearer sk-...
Content-Type: application/json

{
  "model": "gpt-4o-mini",
  "messages": [{"role": "user",
    "content": "Resume este texto..."}]
}
```

Tu agente / n8n / LangChain envia esto

```
# Lo que OpenAI te devuelve (JSON)
200 OK

{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "El texto resume..."
    }
  }]
}
```

OpenAI te devuelve esto

¿Por que importa saber esto?

Cuando conectas OpenAI en n8n o usas LangChain, estas haciendo exactamente esto de forma automatica. La API key es tu 'carnet' para acceder. El modelo es el 'plato' que pides. El JSON de respuesta es lo que comes. Los errores mas comunes: 401 (key invalida), 429 (demasiadas peticiones), 500 (fallo del servidor).

n8n: Workflows en practica



n8n en práctica — 4 workflows para ejecutar hoy

Ya se hizo un RAG básico. Ahora subimos el nivel con flujos agentes reales.

1. AI Agent con Memoria

Un agente con historial de conversación.

Recuerda lo que dijiste antes. El más fácil de levantar — solo necesitas API key de OpenAI.

Trigger: Chat | Tools: LLM + Memory

2. Telegram AI Bot

Escribes desde el movil, el agente responde.

El más impactante en clase. Ves la respuesta en tiempo real en tu teléfono.

Trigger: Telegram | Tools: LLM + Tools opcionales

3. Email Responder con HITL

El agente redacta, tu apruebas antes de enviar.

Conecta con Human-in-the-Loop. El agente propone, un humano valida antes de ejecutar.

Trigger: Gmail | Tools: LLM + Gmail Draft

4. AI Data Analyst

Preguntas en lenguaje natural sobre tus datos.

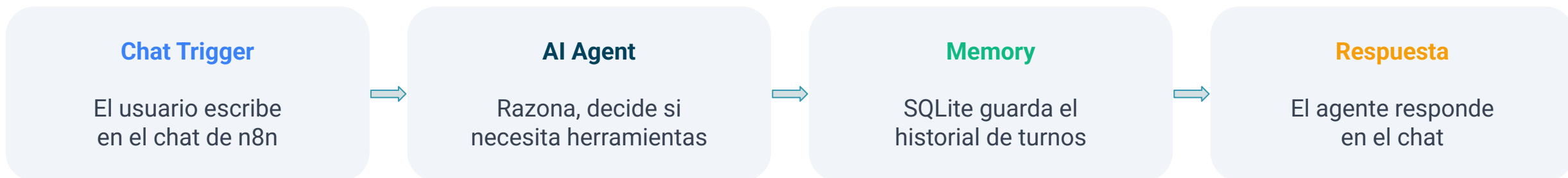
Conecta Google Sheets o un CSV. El agente analiza y responde. Muy relevante para Data Science.

Trigger: Chat | Tools: LLM + Google Sheets

Setup en local: `npx n8n` (o `docker run -it --rm -p 5678:5678 n8nio/n8n`) --> `localhost:5678`

Workflow 1 – AI Agent con Memoria

Sencillo, o necesita configurar apps externas – solo tu API key de OpenAI.



Setup (5 minutos)

1. Abre n8n --> New Workflow --> busca template 'AI Agent'
2. En el nodo AI Agent, pon tu OpenAI API key en Credentials
3. Activa el workflow --> abre el Chat --> escribe cualquier cosa
4. Prueba: 'Me llamo Alex' --> en el siguiente turno: '¿Como me llamo?'

Template oficial: 'AI Agent with Memory (SQLite)' -- n8n.io/workflows

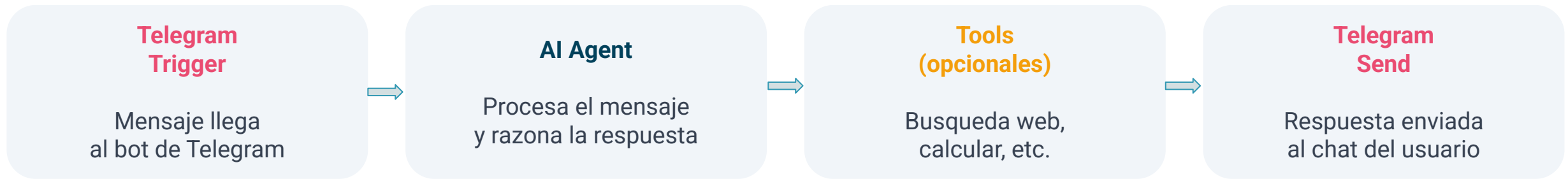
Busca tambien: 'Conversational agent with memory' -- hay varias variantes

Que demostrar

1. El agente recuerda el nombre entre turnos -- sin memory no lo hace
2. Abre el nodo de memoria y muestra como guarda el historial en SQLite
3. Borra la memoria y repite -- el agente ya no recuerda. Impacta.

Workflow 2 – Telegram AI Bot

Escribes desde el móvil y el agente responde en tiempo real.



Setup Telegram (10 minutos)

1. Abre Telegram --> busca @BotFather --> /newbot
2. Ponle nombre y username (acaba en _bot)
3. BotFather te da un TOKEN -- copialo
4. En n8n: Telegram Trigger --> pega el token en Credentials
5. Activa el workflow --> escribe al bot desde Telegram

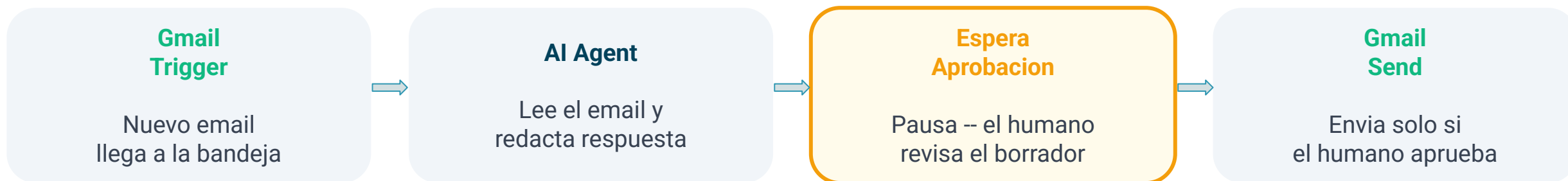
Setup n8n (5 minutos)

Template: 'Telegram AI Chatbot'
o 'Telegram bot with AI Agent'

Nodos: Telegram Trigger --> AI Agent --> Telegram
Añade memoria para que recuerde el hilo

Workflow 3 – Email Responder con Human-in-the-Loop

El agente redacta la respuesta, pero un humano la aprueba antes de enviarla.



Setup (15 minutos)

1. Template: 'AI email responder' o 'Gmail AI auto-reply'
2. Conecta cuenta Gmail en n8n Credentials (OAuth)
3. El nodo 'Wait' pausa el flujo -- n8n te manda el borrador para aprobar
4. Apruebas desde la interfaz de n8n o por email -- el flujo se reanuda

Por que este workflow conecta con lo que hemos visto

Es el mismo concepto de Human-in-the-Loop que vimos en LangGraph.
El nodo 'Wait' de n8n = interrupt_after en LangGraph.
La diferencia: n8n lo hace sin código. En LangGraph tienes control total.

Workflow 4 – AI Data Analyst

Preguntas en lenguaje natural sobre tus datos. El agente consulta, analiza y responde.



Ejemplos de preguntas que puede responder

'¿Cuanto vendimos en Q1 vs Q2?'

'¿Que producto tiene el margen mas alto?'

'Dame los 5 clientes que mas compraron este mes'

'¿Hay alguna anomalia en las ventas de marzo?'

Setup (10 minutos)

Template: 'Chat with Google Sheets'
o 'AI Agent with Spreadsheet tool'

Conecta Google Sheets con OAuth.
Alternativa sin setup: sube un CSV y usa el nodo 'Read File'.

Por que es relevante para Data Science

Es Text-to-SQL / Text-to-Data sin escribir una linea. El agente razona sobre los datos igual que lo haciamos en el Lab 2 con SQL -- pero ahora el usuario no sabe SQL.

Reto: OpenAI Agent Builder



Core

 Agent

 End

 Note

Tools

 File search

 Guardrails

 MCP

Logic

 If / else

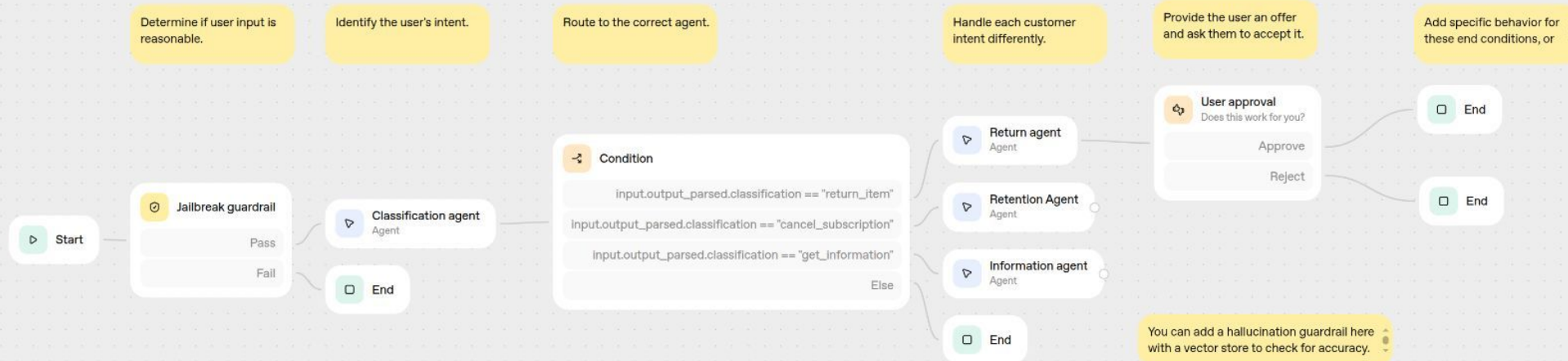
 While

 User approval

Data

 Transform

 Set state



Reto: n8n avanzado



Objetivo

Agente Market Researcher

- **Input:** nombre + categoría de producto
- **Output:** informe ejecutivo de mercado (Markdown)
- **Stack:** Web Search + Scraping + LLM
- **Objetivo de tiempo:** LLM < 60 segundos

Workflow n8n

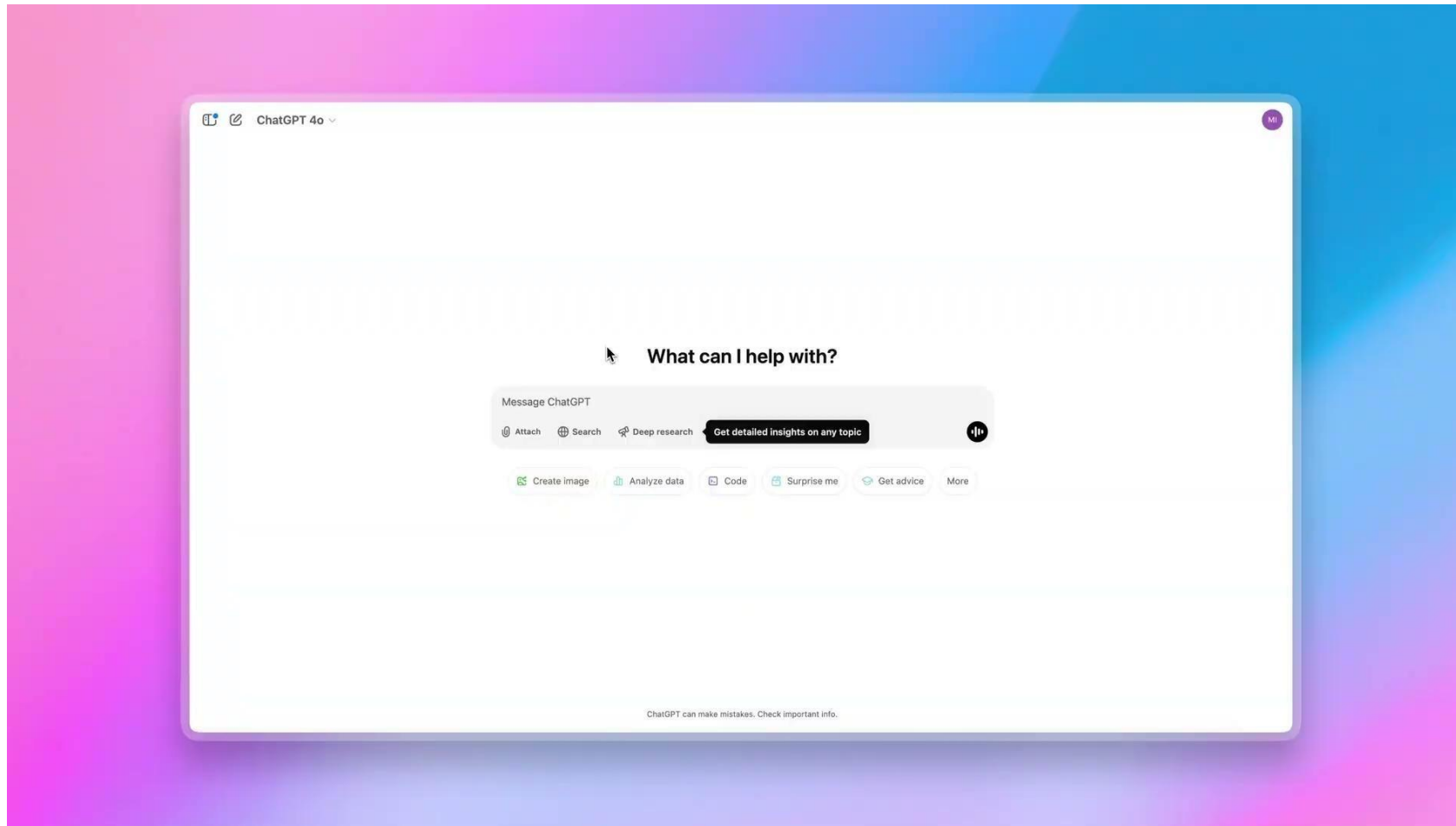
1. **Trigger** (formulario / webhook)
2. **LLM Planner** — descompone la tarea
3. **Tools** — WebSearch + Scraping
4. **Síntesis final** — LLM agregador
5. **Output** — Markdown / Email

Memoria opcional en Postgres

Ejemplos de Agentes

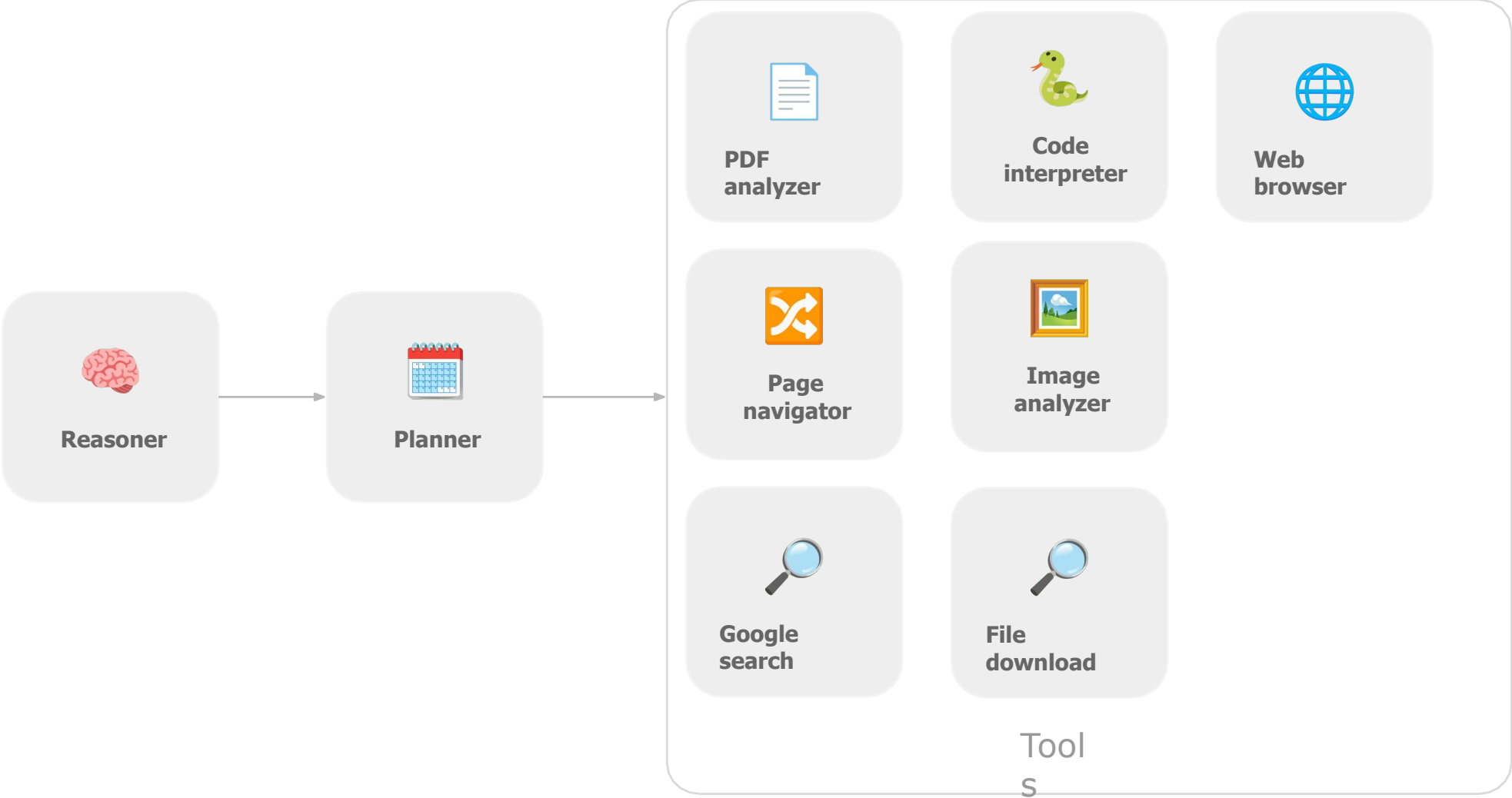


Investigación: OpenAI Deep Research



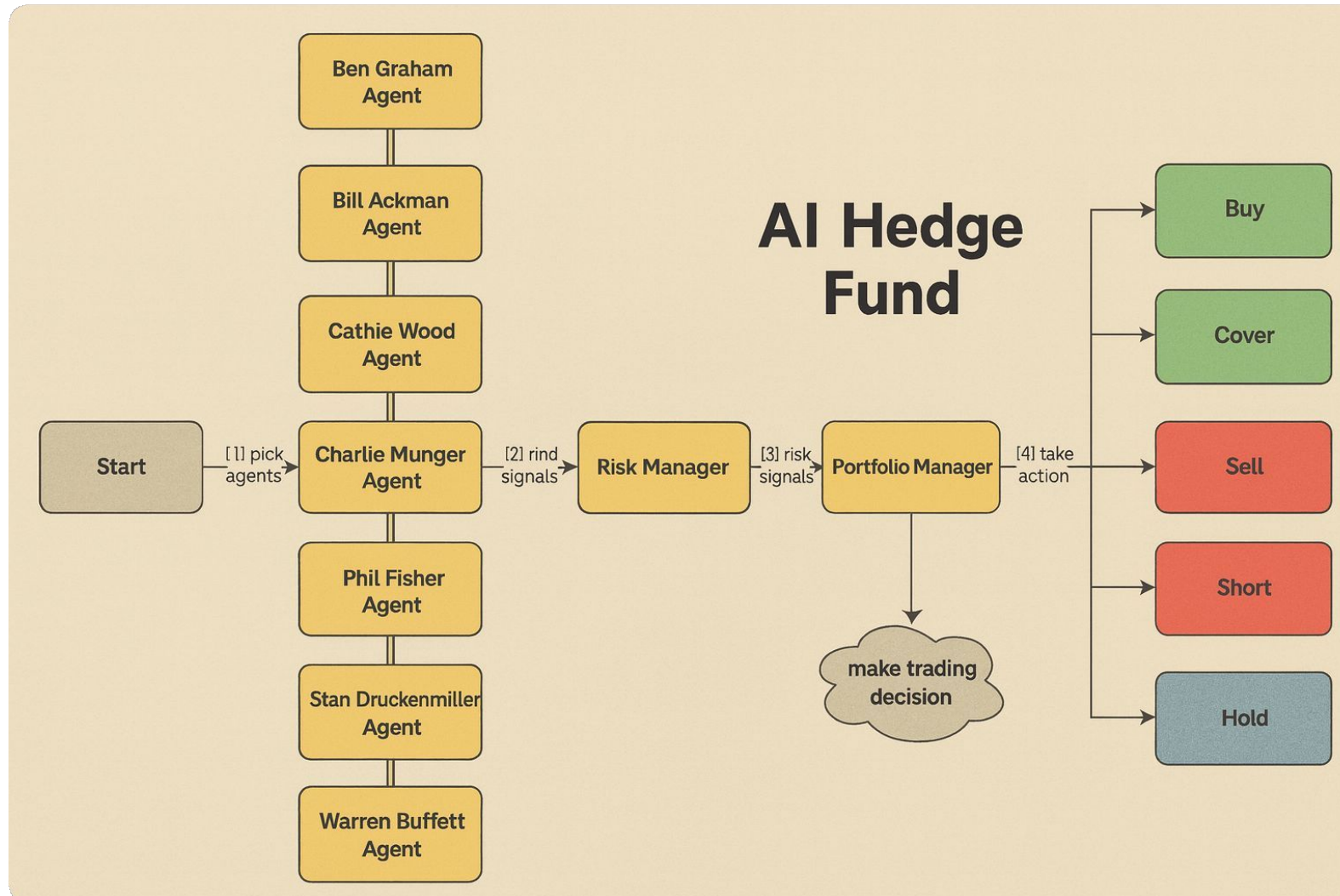
<https://openai.com/es-ES/index/introducing-deep-research/>

- **Agente** que permite realizar **búsquedas en “profundidad”** en internet para resolver desafíos
- Aplica **chain of thought** para descomponer el problema
 - Descompone el problema en un plan y se vale de diferentes herramientas para resolverlo
- **Gran interés** (Feb 2025) en este área
 - Grok, Gemini, Perplexity tienen soluciones similares
- **¿Cómo se evalúan?**
 - Existen algunos retos como GAIA:
<https://openreview.net/pdf?id=fibxvahvs3>
 - Leaderboard HuggingFace:
<https://huggingface.co/spaces/gaia-benchmark/leaderboard>



Inversión: AI Hedge fund

<https://github.com/virattt/ai-hedge-fund>



Mandato de inversión

Importe (€)

100000

Horizonte temporal

largo (>5 años)

Perfil de riesgo

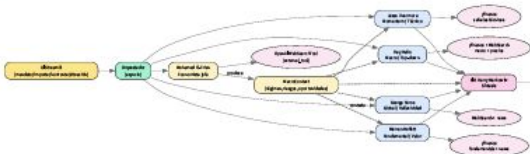
equilibrado

Exclusiones (tickers/sectores separados por coma)

Constructor de Cartera Multiagente

Pipeline: El-Erian (macro vía WebSearch) → Buffett · Dalio · Livermore · Soros (en paralelo) → CIO Markowitz (síntesis) → Optimizador.
Modelo: `got-s.5` · Modelo CIO: `got-s.5`

Arquitectura del flujo



Configuración actual del sistema

Mandato de inversión

Construir una cartera globalmente diversificada con objetivo de revalorización del capital.

Situación de mercado (tu visión)

Desinflación en marcha en países desarrollados, cido de bajadas de la Fed comenzando, crecimiento UE débil, ciclo de capex en IA en curso, riesgo geopolítico de cola en Oriente Medio y Taiwán.

Construir cartera

Cartera generada en 93.9 s

Contexto macro actual (Mohamed El-Erian · WebSearch)

Régimen: Desinflación interrumpida por shock energético + tipos altos por más tiempo + concentración en IA/tech

Crecimiento: El FMI proyecta crecimiento mundial de 3,1% en 2026, con clara bifurcación: EEUU alrededor de 2,3%, eurozona 1,4%, China 5,0% y emergentes 4,4%. La señal central es crecimiento positivo pero frágil: EEUU sigue siendo el motor de los mercados desarrollados, Europa permanece débil y China/Asia sostienen el bloque emergente.

Inflación: La desinflación ha perdido linealidad por el shock energético de Oriente Medio: el IPC de EEUU repuntó a 3,8% interanual en abril de 2026, con

Estado de mercados: La renta variable sigue apoyada por beneficios, liquidez selectiva y capex en IA, pero con valoración y concentración elevadas: SPY cotiza cerca de 739 USD y QQQ cerca de 709 USD el 18 de mayo de 2026; Europa corrige más por energía y crecimiento débil, con FEZ en 64,95 USD y exposición España/EWP en 55,63 USD. Emergentes están bajo presión táctica por dólar, petróleo y riesgo geopolítico —EEM en 65,07 USD—; Brent ronda 111 USD/barril y el oro mantiene fuerte prima defensiva, reflejada en GLD cerca de 417 USD.

#1 tool_call → get_quote

`{"ticker": "SPY"}`

#2 tool_call → get_quote

`{"ticker": "QUAL"}`

#3 tool_call → get_quote

`{"ticker": "IEF"}`

#4 tool_call → get_quote

`{"ticker": "TIP"}`

#5 tool_call → get_quote

#6 tool_call → get_quote

#7 tool_call → get_quote

`{"ticker": "EEM"}`

#8 tool_call → get_quote

#9 tool_call → get_price_history

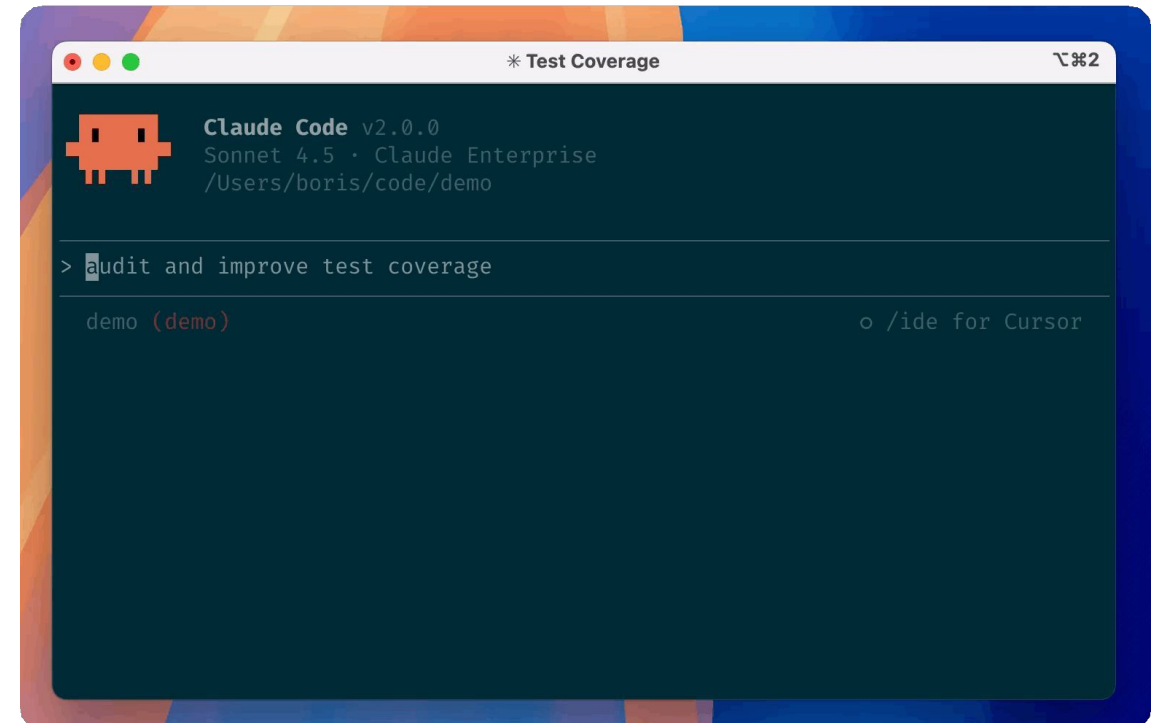
`{"ticker": "SPY", "period": "1y"}`

#10 tool_call → get_price_history

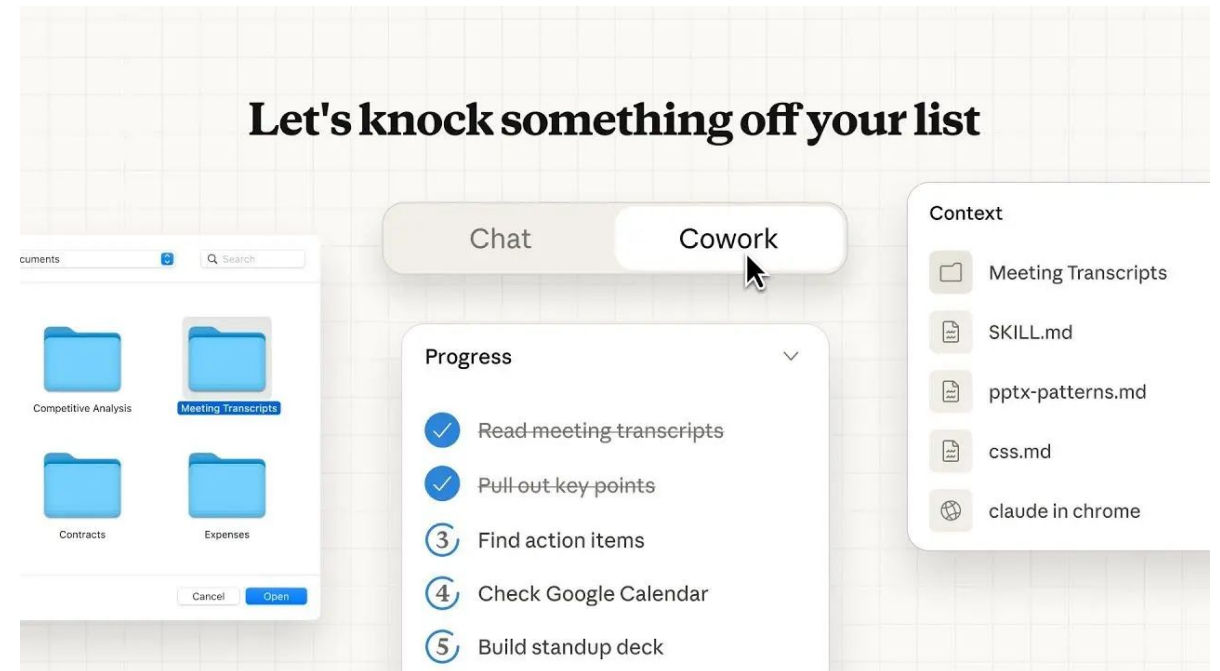
#11 tool_call → get_price_history

<https://github.com/anthropics/claude-code>

- Agente de programación:
 - Refactoriza, corrige, explica código, gestiona Git y responde sobre proyectos.
 - Extra el contexto inteligente de la base de código.
 - Ciclo "Planificar-Actuar-Verificar": A diferencia de un simple chat, el agente planifica la tarea, ejecuta comandos de terminal, edita archivos y corre tests para validar su trabajo.
- Características:
 - Integración local en terminal y plugins.
 - Automatiza edición, commits y pruebas.
 - Integración MCP, Skills y Plugins

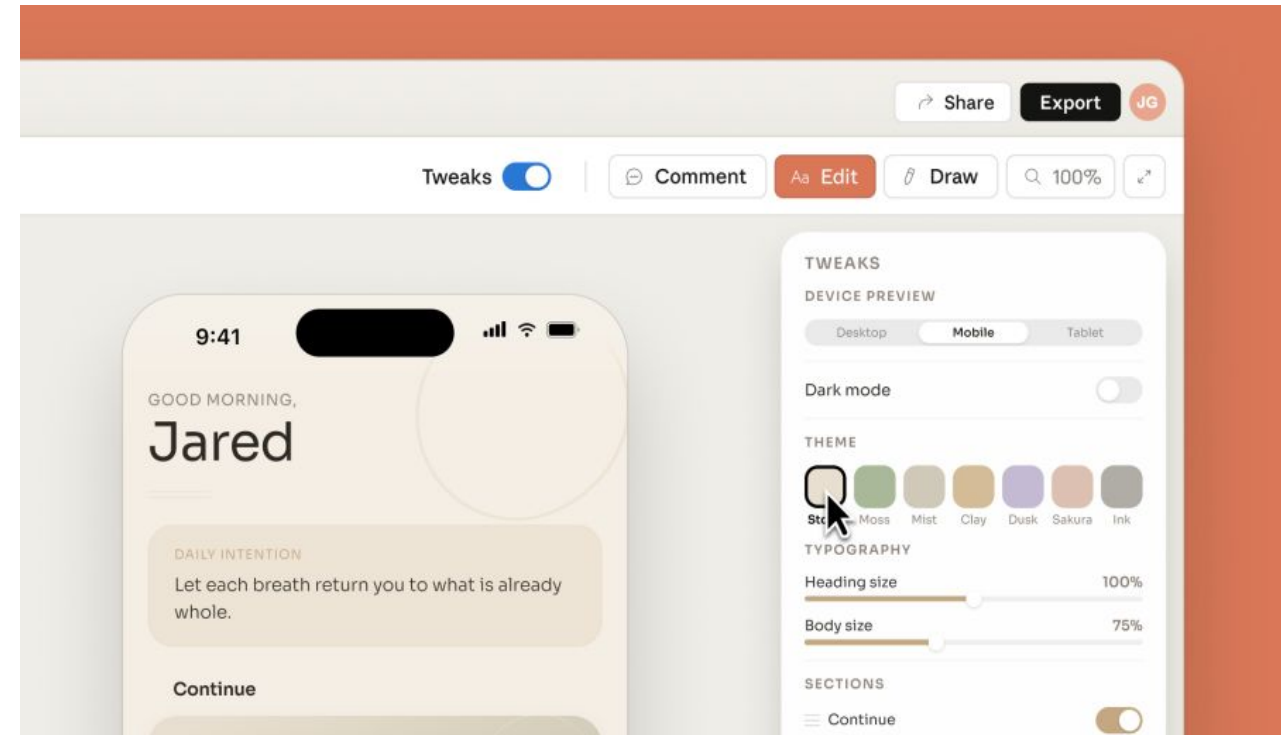


- Agente de trabajo local
- Características
 - Ejecución autónoma: descompone objetivos en pasos y los realiza sin supervisión continua.
 - Acceso a archivos: puede leer, crear, editar y organizar contenido local con permisos explícitos.
 - Enfoque no-code: potencia capacidades “agentic” sin requerir programación.



<https://www.anthropic.com/news/claude-design-anthropic-labs>

- Agente de diseño visual
- Características
 - Crea diseños, prototipos, slides y one-pagers a partir de una descripción
 - Permite iterar mediante conversación, comentarios y edición directa
 - Genera varias alternativas visuales rápidamente
 - Puede aplicar guías de marca o design systems del equipo
 - Facilita el paso de idea → prototipo → material compartible.



<https://claude.com/blog/cowork-research-preview>



Andrej Karpathy ✓

@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, **I don't read the diffs anymore**. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. **It's not too bad for throwaway weekend projects**, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

[Traducir post](#)

12:17 a. m. · **3 feb. 2025** 5,1 M Visualizaciones

1 mil

5 mil

30 mil

15 mil



www.ebiseducation.com



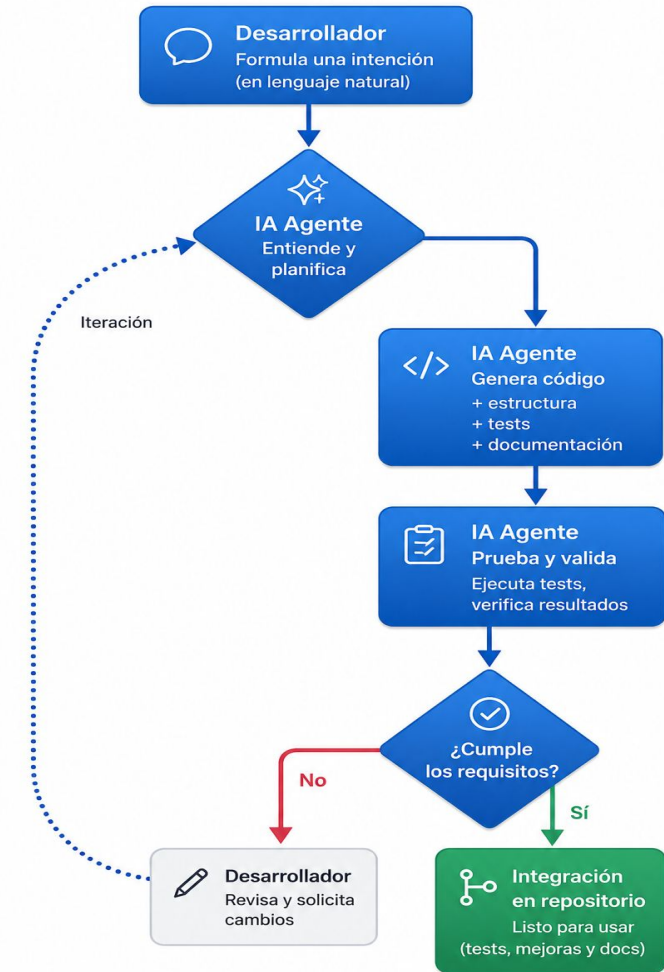
¿Qué es Vibe Coding?



Vibe Coding

Desarrollo asistido por IA donde el desarrollador describe lo que quiere en lenguaje natural, y el agente IA genera, refina y prueba el código.

- El foco pasa de "teclear líneas" a "mente/prompt → agente genera → humano revisa/refina".
- **Ideas clave:** diálogo, feedback iterativo, especificación mínima inicial, más experimentación.



Vibe Coding en Números

La adopción es masiva.

92%

de devs de EE.UU.
usan herramientas IA de coding a
diario

46%

del código nuevo en GitHub
lo genera la IA

87%

de empresas Fortune 500
ya han adoptado vibe coding

<https://tech.co/news/survey-developers-using-ai-more-trust-less>

LA PARADOJA DE LA CONFIANZA

77%

favorabilidad de desarrolladores (2023)



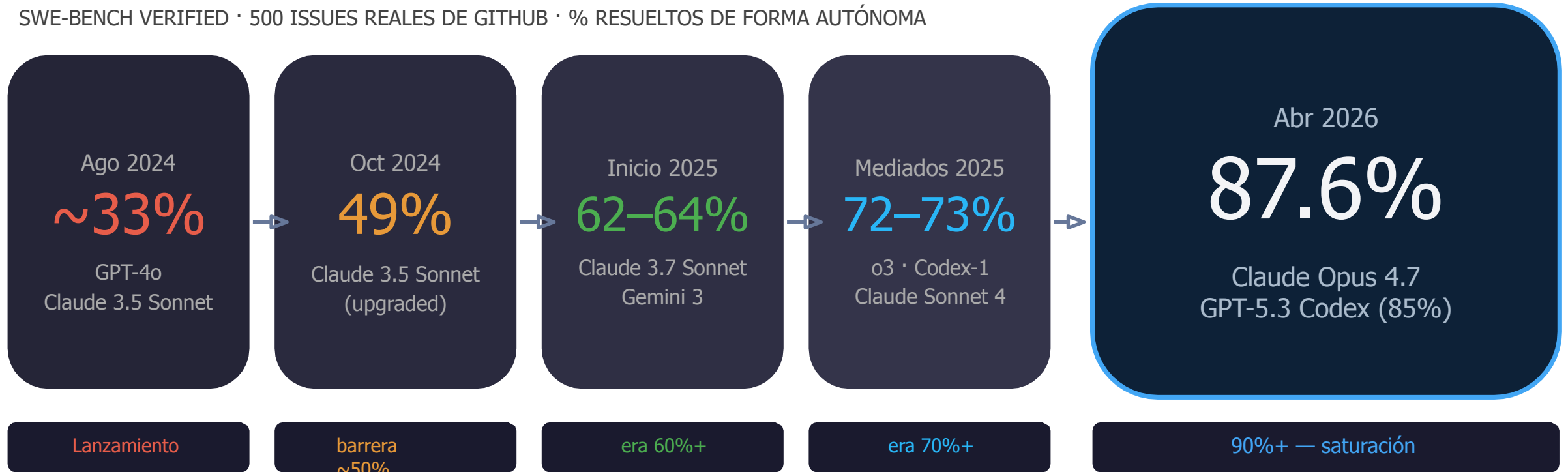
60%

favorabilidad de desarrolladores (2025)

*La industria usa
intensivamente algo en lo
que cada vez
confía menos.*

El Auge del AI Coding

SWE-BENCH VERIFIED · 500 ISSUES REALES DE GITHUB · % RESUELTOS DE FORMA AUTÓNOMA



En 20 meses, la IA pasó de resolver 1 de cada 3 issues reales de programación a casi 9 de cada 10.

<https://benchlm.ai/benchmarks/sweVerified>



No son “apps 100% vibe-coded”: son agentes integrados en el ciclo real de desarrollo, con plataforma, tests y revisión humana.



Spotify

Migraciones de código a escala

650+

PRs/mes generadas por el agente
y mergeadas en producción

90%

menos tiempo en migraciones
complejas frente a trabajo manual

Patrón: prompt/spec → agente → PR → CI → merge humano

De escribir código a orquestar agentes: Spotify usa agentes en background para convertir trabajo repetitivo de ingeniería en PRs revisables, liberando capacidad para producto, arquitectura y deuda técnica.

<https://claude.com/customers/spotify>

UBER Uber

GenAI en code review y productividad

>90%

de ~65.000 diffs/semana
analizados

75%

comentarios marcados como
útiles

>65%

comentarios publicados
atendidos

Patrón: diff → agente revisa → feedback → humano decide → merge

uReview aumenta la capacidad de ingeniería revisando automáticamente diffs reales: más feedback, menos cuello de botella en code review y decisión final humana.

<https://www.uber.com/us/en/blog/ureview/>

Takeaway: el valor enterprise está en reducir toil — migraciones, reviews, tests, deuda técnica — no en “magia autónoma”.



No sólo acelera el código: también está cambiando cómo se dimensionan los equipos.



TECNOLOGÍAS >

Capgemini anuncia un ERE en España por la incidencia de la inteligencia artificial

La consultora, que cuenta con una plantilla de más de 11.000 profesionales en territorio nacional, alega un “entorno operativo cada vez más incierto”

<https://elpais.com/economia/2026-04-10/capgemini-anuncia-un-ere-en-espana-por-la-incidencia-de-la-inteligencia-artificial.html>

La empresa justificó el ajuste por el impacto de la IA y la necesidad de adaptarse a nuevas demandas del mercado. El preacuerdo redujo la afectación inicial de 748 a 710 personas.



Amazon anuncia 16.000 despidos más en plena reorganización por la IA

La multinacional vuelve a ajustar su plantilla tras la salida de 14.000 empleados en octubre, mientras justifica razones de «organización» frente a los cambios impuestos por tecnologías como la Inteligencia Artificial

<https://www.abc.es/economia/amazon-anuncia-16000-despidos-plena-reorganizacion-ia-20260128130252-nt.html?ref=https%3A%2F%2Fwww.abc.es%2Feconomia%2Famazon-anuncia-16000-despidos-plena-reorganizacion-ia-20260128130252-nt.html>

la IA no solo reduce tareas repetitivas; también cambia el cálculo de plantilla. Si los agentes aumentan la productividad, algunas funciones corporativas dejan de crecer al mismo ritmo.



Meta despide a 8000 empleados, a medida que aumenta el uso de la IA

La agitación en Meta —propietaria de Facebook, WhatsApp e Instagram— ofrece una visión cercana de los despidos en la era de la inteligencia artificial.

<https://www.nytimes.com/es/2026/05/20/espanol/negocios/meta-facebook-despidos-ia.html>

la IA funciona como doble palanca: automatiza parte del trabajo y, a la vez, absorbe inversión. Las compañías recortan áreas existentes para financiar equipos y productos AI-first.

Tranquilos!! la IA no elimina el talento tech; elimina tareas repetitivas. El perfil que gana es quien sabe definir bien el problema, guiar agentes, revisar resultados y convertirlos en software seguro y útil.

Vibe Coding va más allá de escribir software.



Presentaciones

Generar y editar decks con prompts en lenguaje natural



Pipelines ETL

Construir flujos de extracción, transformación y carga de datos



Comandos del sistema

Automatizar scripts de shell, tareas DevOps y pipelines CI/CD



Mantenimiento de sistemas

Monitorizar sistemas, predecir fallos y gestionar infraestructura



Documentación

Crear docs técnicas, informes y specs automáticamente



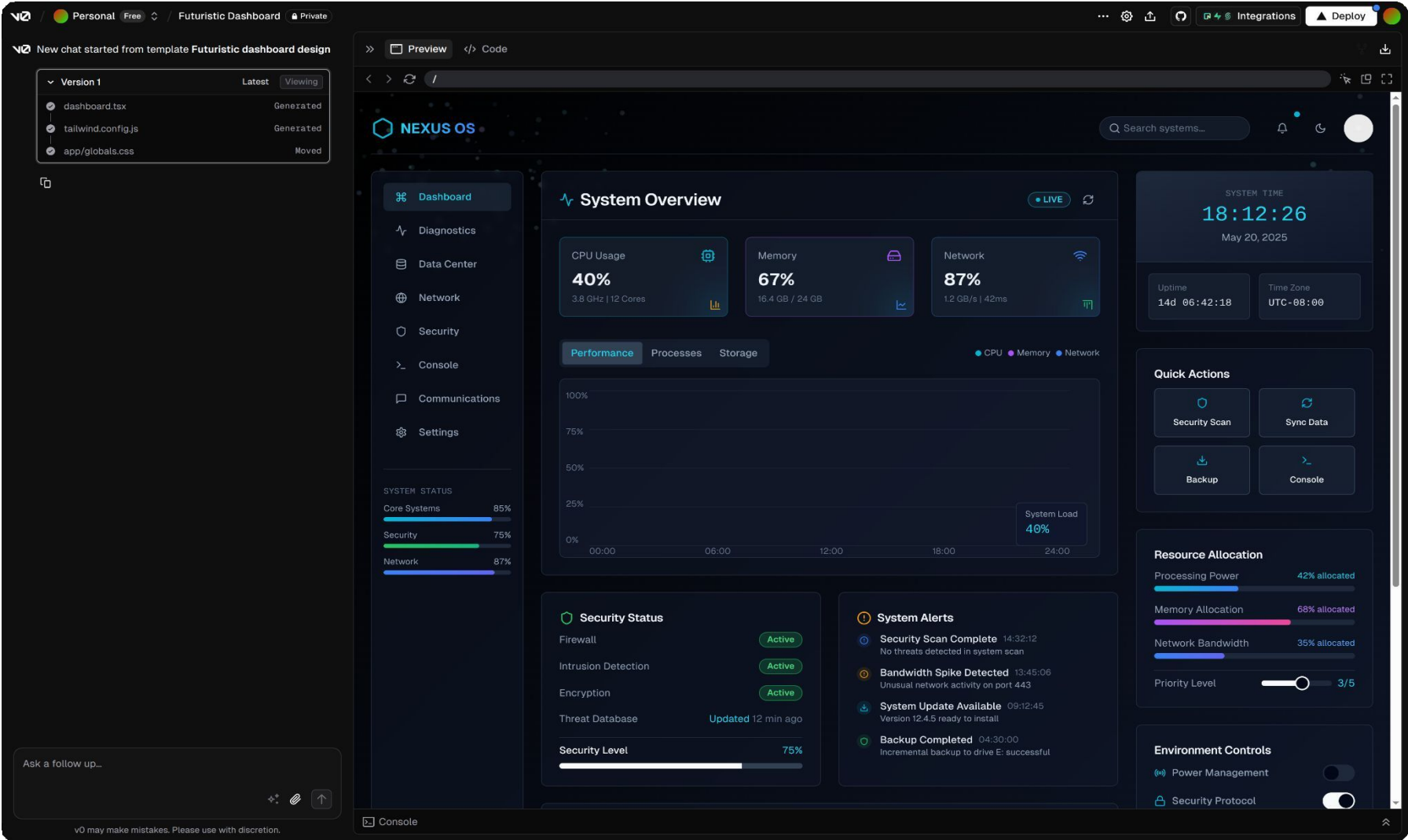
Análisis de datos

Generar consultas SQL, dashboards e informes visuales



Herramientas low-vibe Coding





← Back to start

Gemini

Migrated "DDoS Simulator & Tutor"

Checkpoint

esta bien el DDOS este?

Gemini 3.5 Flash • Running for 92s

Implementing Chart Optimization

DDoS Simulator & Tutor

Remix Share Publish

Preview Code

DDoS Lab

Simulador Interactivo de Ataques de Red

ESTADO DEL SERVIDOR

OPERATIVO

Panel de Control

Intensidad del Ataque (Botnet) 12%

Defensa (WAF / Firewall) DESACTIVADO

Tráfico Legítimo Tráfico Ataque

Asistente IA de Seguridad

Hola, soy tu asistente de seguridad NetGuard. Pregúntame qué es un DDoS o cómo detenerlo.

TRÁFICO EN TIEMPO REAL

0% CARGA



Beneficios, Riesgos y Seguridad



Beneficios del Vibe Coding

- **Prototipado más rápido:** Menos boilerplate, más lógica.
- **Más experimentación:** Probar ideas con bajo "coste" manual.
- **Documentación integrada:** El agente explica sus propuestas.
- **Desarrollo más democrático:** Colaboración activa de roles (sin reemplazar al ingeniero).
- **Mejor comprensión y onboarding rápido:** Agentes facilitan la Integración de nuevos miembros.
- **Cambio de valor:** De programar detalles a dirigir, entender y orquestar.





I Am Devloper ✓

@iamdevloper

vibe coding, where 2 engineers can now create the tech debt of at least 50 engineers

How AI Vibe Coding Is Destroying Junior Developers' Careers

New research shows developers think AI makes them 20% faster but are actually 19% slower. Vibe coding is creating unemployable pseudo-developers who can't debug or maintain code.



Written by
Kaustubh Saini



Edited by
Jaya Muvania



Reviewed by
Kaivan Dave

Updated on
Jul 16, 2025

Read time
9 Min Read

<https://www.finalroundai.com/blog/ai-vibe-coding-destroying-junior-developers-careers>

TODOS MIENTEN.

EL LADO OSCURO DEL VIBE CODING DEL QUE NADIE HABLA



En redes sociales, el "vibe coding" se ve fácil.
Alguien escribe dos líneas de código. Publica una captura. Aplausos instantáneos.



Pero esta es la verdad:

- La mayoría no funciona.
- La mayoría no escala.
- La mayoría se rompe en producción.

LA INGENIERÍA REAL NO TRATA DE "VIBES"

Se trata de sistemas que no se caen a las 2 AM.
Se trata de pipelines, monitoreo, depuración y confiabilidad a largo plazo.



EL VIBE CODING ES DIVERTIDO PARA CONTENIDO

Pero es peligroso como mentalidad. Si quieres construir algo real:



ENVÍA



RÓMPELO



ARRÉGLALO



REPÍTELO

Riesgos del Vibe Coding

- **Dependencia tecnológica:** se pierde comprensión del código generado.
- **Errores/Alucinaciones:** el código generado puede ser sutilmente defectuoso.
- **Pérdida de trazabilidad:** difícil auditar decisiones y fuentes (riesgo con información sensible).
- **Desalineación de estándares:** los agentes pueden ignorar guías de estilo o arquitectura.
- **Falsa productividad:** mayor velocidad no garantiza calidad o sostenibilidad.
- **Menor aprendizaje junior:** delegar reduce la práctica y comprensión de fundamentos técnicos.



1.7x más vulnerabilidades graves en código desarrollado por IA vs escrito por humanos

<https://www.businesswire.com/news/home/20251217666881/en/Code-Rabbits-State-of-AI-vs-Human-Code-Generation-Report-Finds-That-AI-Written-Code-Produces-1.7x-More-Issues-Than-Human-Code>

45% dg las muestras generadas por IA contienen vulnerabilidades del OWASP Top-10

<https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/>

69 vulnerabilidades (6 críticas) encontradas en 15 apps idénticas construidas con 5 herramientas

<https://blog.tenzai.com/bad-vibes-comparing-the-secure-coding-capabilities-of-popular-coding-agents/>

63% de desarrolladores pasan más tiempo depurando código de IA que escribiéndolo ellos mismos

<https://wowhow.cloud/blogs/dark-side-vibe-coding-debugging-ai-code-problems-2026>



Qué sale a los proveedores de LLMs y cómo proteger código sensible.

¿Qué sale fuera?

Código fuente, prompts y respuestas.

Personales: Se usan para entrenamiento.

Enterprise: Protegidos por contrato.

RGPD / NIS2 — Seguridad

- Prohibido subir PII, contratos o secretos.
- Secret scanning obligatorio en pre-commit.
- Logs sin datos personales.

Alternativas Seguras

Workspace Ent. (EU Isolation) y **Vertex AI** (Europe-west). Modelos **On-prem** (Llama, Mistral) y Proxies de redacción de PII.

Recomendación Práctica

Sandbox / R&D: Cuenta personal OK con repos públicos.

Producción: Workspace Ent. o Vertex AI (Región EU). **Nunca** cuenta personal con código de cliente.



Herramientas de Coding Agents



Herramientas AI Coding

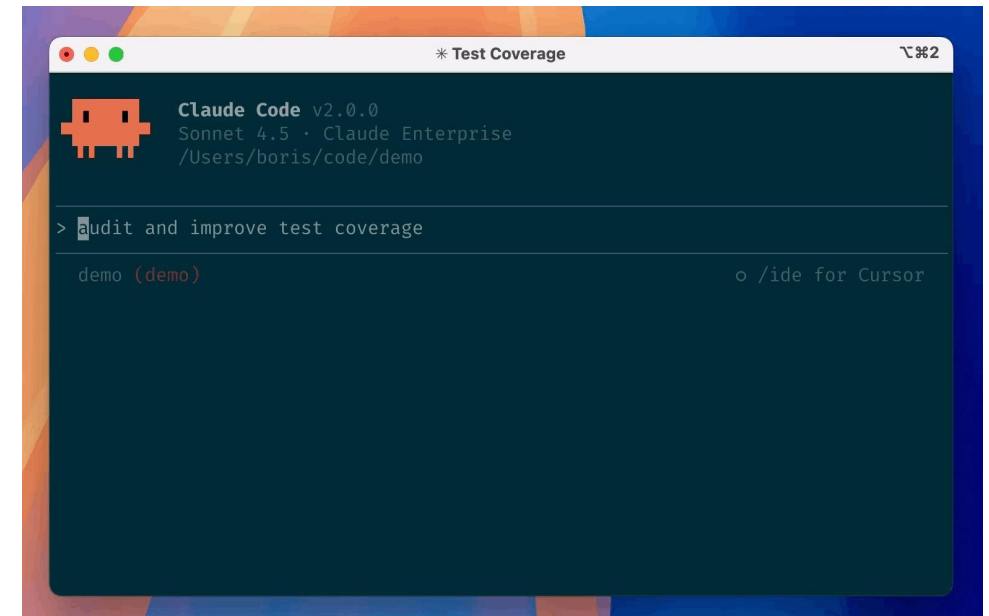
Mayo 2026

Herramignta	Free Tier	Precio	Plugins / MCP	Puntos fuertes	Limitaciones	IDE nativo
Claude Code Anthropic	No	Pro: US\$20/mes Max: US\$100–200/mes	Sí – Plugins y MCP	Coding agéntico profundo en terminal, sistema de plugins (skills, hooks, MCP), modelos Opus 4.7 y Sonnet 4.6	Sin free tier para Code (mín US\$20/mes), solo CLI sin IDE gráfico	No
Gemini CLI Google	Sí	AI Pro: US\$19.99/mes AI Ultra: US\$125/mes	Sí – extensiones de IDE y MCP	Contexto 1M tokens, free tier 1000 req/día, modelos Gemini 3, marketplace de extensiones	Solo CLI (sin IDE), razonamiento complejo aún por detrás de Claude Opus 4.7	No
Codex OpenAI	No	Plus: US\$20/mes Pro: US\$100–200/mes	Sí – CLI, Slack, IDE, MCP	Codex CLI + IDE + Cloud, cliente y servidor MCP, integración con Agents SDK, alta precisión	Sin free tier, cuotas en ventanas de 5h, MCP consume contexto rápido	No
Cursor AI Cursor	Sí	Pro: US\$20/mes Pro+: US\$60/mes Ultra: US\$200/mes	Sí – multi-modelo y MCP	IDE AI-native con modo Agent, marketplace de plugins (MCP+skills+hooks), MCP Apps interactivas	Precios por créditos pueden confundir, caro para power users en modelos frontier	Sí
Windsurf IDE Cognition AI	Sí	Pro: US\$20/mes Max: US\$200/mes	Parcial – integración nativa	Agente Cascade con contexto codebase-wide, modelos SWE-1.5, Codemaps	Adquirido por Cognition AI (dic 2025), nuevos precios desde abr 2026, integración Devin aún pendiente	Sí



<https://github.com/anthropics/claude-code>

- **Agente de coding:**
 - Refactoriza, corrige, explica código, gestiona Git y responde preguntas del proyecto.
 - Extrae contexto inteligente del codebase.
 - Loop "Plan-Act-Verify": a diferencia de un chat simple, el agente planifica la tarea, ejecuta comandos en terminal, edita archivos y corre tests para validar su trabajo.
- **Características:**
 - Integración con terminal local y plugins.
 - Automatiza edición, commits y testing.
- **Instalación:**
 - `npm install -g @anthropic-ai/claude-code`



- <https://github.com/google-gemini/gemini-cli>
- Herramienta CLI de Google para usar modelos Gemini directamente desde el terminal.
 - Permite generar, revisar y ejecutar código, comandos o análisis contextual.
 - Tiene un free tier tanto para un modelo state-of-the-art como uno flash
 - Herramientas: Grounding con Google Search, operaciones de archivos, comandos de shell, fetch web.
- Instalación: Compatible con macOS, Linux y Windows (requiere Node.js v20+).
 - `npm install -g @google/gemini-cli`
 - Ejecuta gemini en la carpeta de tu proyecto.
- Configuración:
 - Por defecto usa archivos GEMINI.md



```
> GEMINI

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. ./help for more information.

> write a short paragraph about why Gemini CLI is awesome

+ I will start by searching the web for "Gemini CLI" to understand its main features and purpose. Following that, I'll examine the README.md file for a project-specific overview. I will then consult docs/index.md and docs/cli/commands.md to gain a deeper understanding of its capabilities. Finally, I will write a short paragraph summarizing why Gemini CLI is awesome.

- GoogleSearch Searching the web for: "Gemini CLI features and purpose"

: Uncovering Gemini's Awesome (esc to cancel, 21s)

Using 3 GEMINI.md files

~/code/gemini-cli (release*)    no sandbox (see /docs)    gemini-2.5-pro (99% context left)
```



Free tier, contexto 1M tokens, CLI open-source de Google.

 **Instalación**
npm install -g
@google/gemini-cli

 **Free tier**
1.000 requests / día
(cuenta personal de Google)

 **Contexto**
ventana 1M tokens
Gemini 3

```
# Get going
gemini # interactive REPL

# One-shot prompt
gemini -p "explain this repo in 3 sentences"

# Pick a model + write to files
gemini -m gemini-2.5-pro -p "refactor src/api.py and add tests"

# Pipe stdin into the prompt
cat error.log | gemini -p "diagnose this stack trace"

# Slash comandos inside the REPL
/help      /tools    /memory show    /chat    /quit
```



1. Instalar

Requiere Node.js ≥ 20 :
`npm install -g @google/gemini-cli`

Comprueba: `gemini --version`

2. Primer arranque

Lanza desde la carpeta del proyecto:
`gemini`

Abre REPL interactivo y pide login.

3. Opciones de autenticación



Cuenta Google personal — free tier 1000 req/día. Datos pueden usarse para training. Sandbox/personal OK, NO para código de empresa.



Google AI Pro (\$19.99/mes) — más cuota, datos no usados para training (contractual).



Workspace Enterprise — data isolation EU, controles SSO/SAML. Recomendado para Empresas.



Vertex AI - API Vertex AI — API empresarial, región europe-west, billing GCP, audit logs. Mejor para producción.



Gemini CLI: práctica - GEMINI.md: contexto del proyecto (2/4) **EBIS** Business Techschool

El agente lo lee en cada turno. Mantén < 300 líneas. Qué / Por qué / Cómo.

```
# GEMINI.md

## Project
API REST para gestión de incidencias de la CRA.

## Stack
Python 3.12 · FastAPI · PostgreSQL · Docker · Kafka
GitLab interno · pre-commit (ruff, mpy, detect-secrets)

## Run / test / lint
uv sync && uv run pytest -q
uv run ruff check . && uv run mpy src

## Conventions
- Endpoints en src/api/v1/
- Acceso a BD solo vía src/repo/ (sin SQL crudo)
- Logs SIN PII de clientes (usar log_safe())
```

✓ Buenas Prácticas

- Define claramente el stack
- Incluye comandos de test
- Usa español para errores

⊘ Restricciones (Don't)

- No tocar /legacy sin el lead
- No añadir dependencias solo
- Cero PII o secretos en logs



Conecta Gemini a tus sistemas internos. Config en ~/.gemini/settings.json



Servidores MCP útiles

- **GitLab / GitHub:** PRs, issues, code review
- **Jira / Linear:** Gestión de tickets
- **SQL / BigQuery:** Consultas a bases de datos
- **Slack:** Notificaciones en tiempo real
- **MCP Custom:** APIs internas de la empresa



Configuración JSON

```
{
  "mcpServers": {
    "gitlab": {
      "command": "npx",
      "args": ["@modelcontextprotocol/server-gitlab"],
      "env": {"GITLAB_TOKEN": "..."}
    }
  }
}
```



Reglas y Seguridad

- **Eficiencia:** Cada MCP consume tokens. Activa sólo los estrictamente necesarios.
- **Credenciales:** Usa variables de entorno. *Nunca* incluyas tokens en archivos JSON commiteados.
- **Auditoría:** Revisa el código de los MCP de terceros. Son vectores de exfiltración de datos.
- **Control REPL:** Usa `/mcp` para listar y gestionar servidores activos en caliente.



Cheat-sheet de comandos REPL y one-shot. Pruébalos sobre un repo público o de sandbox.

Dentro del REPL

```
$ gemini

/help          # ayuda
/tools         # tools disponibles (MCP)
/memory show   # ver GEMINI.md cargados
/chat         # historial de la sesión
/model gemini-3-pro
/mcp          # gestiona servers MCP
/clear        # limpia contexto
/quit         # salir
```

One-shot (scriptable)

```
# resumen del repo
gemini -p "explica este repo en 3 frases"

# diagnosticar logs
cat error.log | \
  gemini -p "diagnostica este stack trace"

# refactor + tests
gemini -p "refactoriza src/api.py y añade
tests pytest"

# elegir modelo concreto
gemini -m gemini-3-pro -p "..."
```

Nota: **REPL** significa Read–Eval–Print Loop (leer → evaluar/ejecutar → imprimir resultado → repetir)

Es un modo interactivo de usar una herramienta o lenguaje. En vez de lanzar un comando completo y terminar (one-shot), entras en una sesión donde escribes instrucciones una a una y la herramienta responde.

Buenas Prácticas



Define un contexto claro:

- Documenta el proyecto en AGENTS.md / CLAUDE.md / GEMINI.md.
- Incluye entorno, dependencias, comandos y estilo de código.
- Añade decisiones arquitectónicas y patrones aprendidos.
- Usa archivos anidados por carpeta: /frontend, /api, /docs...
- Declara skills, sub-agentes y herramientas disponibles.
- Configura CI/CD, tests, linting y formateo.
- Actualiza el contexto al final de cada sesión productiva.



El contexto no es un prompt: es documentación viva para humanos y agentes.

Ejemplo Claude.md

```
# CLAUDE.md
```

```
## Objetivos del proyecto
```

- Crear un blog personal con enfoque en tecnología y desarrollo
- Optimizar la velocidad de carga y experiencia de usuario
- Mantener un diseño minimalista y accesible
- Facilitar la categorización y búsqueda de contenido

```
## Tecnologías utilizadas
```

- Hugo como generador de sitios estáticos
- HTML5, CSS3 y JavaScript para frontend
- Bear theme como base de diseño
- Nginx para servir el contenido estático
- Traefik como proxy inverso y gestor de SSL
- Docker y Docker Compose para orquestación de servicios

```
## Buenas prácticas
```

- Utilizar imágenes optimizadas (WebP, dimensiones apropiadas)
- Mantener una estructura de carpetas coherente para posts (/año/tema)
- Seguir convenciones de Markdown para contenido consistente
- Priorizar la carga de contenido crítico
- Minimizar dependencias externas para mejorar privacidad
- Aplicar HTTPS en todas las conexiones
- Implementar caché adecuado para recursos estáticos

```
## Patrones aprendidos
```

- Las llamadas a la API externa deben usar el cliente con reintentos
- Los componentes de UI mantienen estado local solo para UX, no para lógica de negocio
- Logs de error incluyen ID de transacción para trazabilidad

```
## Decisiones arquitectónicas
```

- Usamos arquitectura hexagonal (09/03/2025)
- Migrando de Redux a Context+Reducers (15/04/2025)
- Feature flags gestionadas en config/features.ts (02/05/2025)

```
## Comandos importantes
```

- `pnpm dev` - Servidor local (http://localhost:3000)
- `pnpm test` - Suite de pruebas completa
- `pnpm lint` - Verifica estilo y errores potenciales

```
## Pasos a completar en cada nueva feature
```

1. Crear rama feature/nombre-descriptivo desde main
2. Implementar cambios siguiendo las buenas prácticas documentadas
3. Verificar la visualización en dispositivos móviles y escritorio
4. Comprobar rendimiento (Lighthouse score > 90)
5. Solicitar revisión antes de integrar a main
6. Desplegar primero en entorno de staging
7. Validar funcionalidad en staging antes de promover a producción



Menos es más (< 300 líneas). Qué / Por qué / Cómo.

✓ HAZ

- Mapa del proyecto: dónde vive cada cosa
- Stack tecnológico y versiones
- Cómo ejecutar, testear, lintear
- Convenciones y reglas no obvias
- Comandos de verificación

✗ NO HAGAS

- Repetir lo que ya dice el código
- Prosa larga / filosofía
- Docs de API que se desactualizan
- Secrets, tokens, credenciales
- Planes futuros aspiracionales

```
# AGENTS.md
```

Project

Slack bot that summarises support tickets every hour.

Stack: Python 3.12 · FastAPI · PostgreSQL · uv

Ejecuta / test / lint

```
uv sync                                # install deps
```

```
uv run pytest -q                       # run tests
```

```
uv run ruff check .                   # lint
```

```
uv run mypy src                       # type-check
```

Conventions

- All HTTP handlers live in src/api/

- DB access only via src/db/repo.py (no raw SQL elsewhere)

- Error messages always in English

Don't

- Don't bypass the repo layer

- Don't add new top-level deps without asking

- Don't commit anything in /secrets or .env



🎯 ¿Qué son las Skills?

- Archivos markdown (SKILL.md) que extienden Claude con prompts reusables.
- Invoca con /skill-name — Claude también puede auto-cargarlas.
- A nivel de proyecto (.claude/skills/) o personal (~/.claude/skills/).

⚙️ Características clave

- \$ARGUMENTS — acepta parámetros del usuario.
- context: fork — ejecuta en sub-agente aislado.
- allowed-tools — restringe herramientas (p. ej. solo lectura).
- !`command` — inyecta salida de shell en el prompt.
- disable-model-invocation — solo disparo manual.

```
.claude/skills/deploy/SKILL.md
```

```
---  
name: deploy  
description: Deploy the app  
disable-model-invocation: true  
---
```

Deploy to **\$ARGUMENTS** :

1. Ejecuta test suite
2. Build the application
3. Push to deployment target
4. Verify deployment succeeded



Buenas Prácticas II

Planifica antes de ejecutar

- Divide el problema en tareas pequeñas.
- Pide un plan y valida el enfoque antes de construir.
- Haz que el agente documente decisiones, riesgos y supuestos.
- Usa el plan como checklist durante la ejecución.
- Paraleliza agentes para código, tests y documentación.

> haz un plan para integrar django unfold en el proyecto

● Voy a investigar el proyecto actual y analizar cómo integrar Django Unfold en el sistema.

● Plan(Investigate Django Unfold integration)
└ Done (42 tool uses · 56.8K tokens · 3m 39s)

● Perfecto, he completado la investigación sobre la integración de Django Unfold en tu proyecto. Aquí está el plan:

Ready to code?

Here is Claude's plan:

Plan de Integración de Django Unfold

Resumen

Django Unfold es un tema moderno para el admin de Django basado en Tailwind CSS. La integración es de bajo riesgo ya que tu proyecto tiene una estructura limpia y modular.

Pasos de Integración

1. Añadir Dependencia (5 min)

- Añadir `django-unfold==0.40.1` a `/backend/pyproject.toml`
- Ejecutar `uv add django-unfold` en el contenedor Django

2. Actualizar Configuración (15 min)

- Modificar `/backend/config/settings/base.py`:
 - Añadir `"unfold"` antes de `"django.contrib.admin"` en `INSTALLED_APPS`
 - Añadir configuración UNFOLD con personalización de sidebar, colores y branding
 - Añadir imports necesarios (`reverse_lazy`, `static`)

3. Actualizar Admin Classes (30 min)

- Cambiar herencia de `django.contrib.admin` a `unfold.admin` en 7 archivos:
- `/backend/mioti_virtual_agent/users/admin.py` - `UserAdmin`
 - `/backend/mioti_virtual_agent/core/admin/call.py` - `CallAdmin` + `CallEventInline`
 - `/backend/mioti_virtual_agent/core/admin/call_event.py` - `CallEventAdmin`
 - `/backend/mioti_virtual_agent/core/admin/contact.py` - `ContactAdmin`
 - `/backend/mioti_virtual_agent/core/admin/faq.py` - `FAQAdmin`
 - `/backend/mioti_virtual_agent/core/admin/program.py` - `ProgramAdmin`

4. Actualizar Widgets JSON (10 min)

- Opción A: Reemplazar `JSONEditorWidget` con `UnfoldAdminTextareaWidget` (recomendado)
- Opción B: Mantener `django-json-widget` y probar compatibilidad



Buenas Prácticas III



Aplica pensamiento crítico

- Revisa y cuestiona cada propuesta del agente.
- Usa roles: arquitecto, reviewer, tester, seguridad.
- Pide alternativas antes de implementar.
- Valida con tests, logs y revisión humana.
- Pregunta: “¿esto lo haría/mantendría un equipo real?”

```
> analiza criticamente los cambios realizados sin commitear
```

- Voy a analizar los cambios no commiteados en tu repositorio. Primero déjame ver qué se modificó exactamente.
- **Bash**(git diff backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py)
└─ diff --git a/backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py b/backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py
index 4d6c849..c060e8d 100644
... +47 lines (ctrl+o to expand)
- **Bash**(git diff .idea/inspectionProfiles/Project_Default.xml)
└─ diff --git a/.idea/inspectionProfiles/Project_Default.xml b/.idea/inspectionProfiles/Project_Default.xml
index bd57690..d78a641 100644
--- a/.idea/inspectionProfiles/Project_Default.xml
... +11 lines (ctrl+o to expand)
- **Read**(backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py)
└─ Read 171 lines

● Análisis Crítico de Cambios Sin Commitear

↩ Resumen de Cambios

Archivos modificados:

- .idea/inspectionProfiles/Project_Default.xml - IDE settings (no relevante)
- backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py - Código Python

Archivos no trackeados:

- 3 archivos de datos (CSV/Excel) con información de programas

● Problemas Críticos

1. Código Muerto (Dead Code)

Se agregaron ~30 líneas de código que nunca se usan:

```
# Estas funciones/variables NO se usan en ningún lugar del archivo:  
_create_program_id_enum()      # Línea 17  
_program_id_enum_cache        # Línea 31  
_get_program_id_enum()         # Línea 34
```

El parámetro program_id en ProgramsCatalogParams:50 sigue siendo str | None, no usa el enum creado.

Impacto: Viola el principio de "Dead Code Cleanup" del proyecto (ver commit reciente 2071929 que eliminó código muerto).

2. Acceso a Base de Datos en Import Time



Buenas Prácticas IV

! Aplica pensamiento crítico

- Evita el “YOLO mode” (auto aceptar todo) en cambios críticos.
- Revisa siempre diffs, tests y comandos ejecutados.
- Guía al agente con TODO/FIXME y prompts inline.
- Limpia el contexto al cambiar de tarea: /clear.
- Usa ramas pequeñas y commits frecuentes.

```
# @gemini: revisa el performance de estas queries
def _create_program_id_enum() → type[Enum]: 1 usage new *
    """Crea dinámicamente un Enum con los external_id de los programas activos desde la base de datos."""
    programs = Program.objects.filter(is_active=True).values_list("external_id", "name")
    choices = {external_id: external_id for external_id, _ in programs}
    if not choices:
        # Si no hay programas, devolver un enum con un valor dummy para evitar errores
        choices = {"_EMPTY": "_EMPTY"}
    return Enum("ProgramIDEnum", choices)
```

```
● Analizando específicamente esta función, encuentro varios problemas técnicos:

● Bash(git diff HEAD backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py | head -40)
  | diff --git a/backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py b/backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py
  | index 4d6c849..c060e8d 100644
  | --- a/backend/mioti_virtual_agent/core/agent/tools/programs_catalog_tool.py
  | ... +37 lines (ctrl+o to expand)

● Search(pattern: "external_id", path: "backend/mioti_virtual_agent/core/models", glob: "*.py", output_mode: "content")
  | Found 11 lines (ctrl+o to expand)

● Revisión Técnica de _create_program_id_enum()

  ● Problemas Críticos

  1. Query Ineficiente

  programs = Program.objects.filter(is_active=True).values_list("external_id", "name")
  choices = {external_id: external_id for external_id, _ in programs}

  ✗ Problema: Está solicitando name desde la BD pero lo descarta con _

  ✓ Corrección:
  programs = Program.objects.filter(is_active=True).values_list("external_id", flat=True)
  choices = {external_id: external_id for external_id in programs}
```



Buenas Prácticas V



Conecta y supervisa

- Aporta contexto real: capturas, logs, trazas y errores.
- Crea comandos reutilizables para tareas del proyecto.
- Conecta herramientas vía MCP: archivos, APIs, docs, issues...
- Limita permisos y revisa acciones sensibles.
- Supervisa cambios antes de aceptar.

> conecta y supervisa

Buenas prácticas para trabajar con agentes en el proyecto

● Aporta contexto real

Capturas, logs, trazas y errores.

📄 captura_pantalla_2025-05-16.png
📄 error_log.txt
📄 trace_2025-05-16.log

● Crea comandos reutilizables para tareas del proyecto.

```
> make dev      # Levanta entorno local
> make test     # Ejecuta tests
> make lint     # Lint del código
> make migrate  # Migraciones DB
```

● Conecta herramientas vía MCP

archivos, APIs, docs, issues...

📁 Archivos (fs) /src, /docs, /config...
🌐 APIs REST, GraphQL, Webhooks...
📖 Docs README.md, Wiki, ADRs...
🐞 Issues GitHub Issues, Pull Requests...

● Limita permisos

y revisa acciones sensibles.

🔒 lectura_archivos	✅ permitido
🔒 escritura_archivos	⚠️ preguntar
🔒 ejecutar_comandos	⚠️ preguntar
🔒 acceso_red	❌ bloqueado
🔒 borrar_datos	❌ bloqueado

● Supervisa cambios

antes de aceptar.

👁️ Ver diff antes de aplicar
🔧 Run tests automáticamente
👤 Revisión humana final

LOGS / ERRORES

ERROR

```
[2025-05-16 10:21:45] ERROR api.views:57
  KeyError: 'user_id'
[2025-05-16 10:21:45] TRACE Request ID: 8f3a...
POST /api/orders/ 500 1.234s
```

COMANDOS REUTILIZABLES

make -l

dev	Levanta entorno local
test	Ejecuta tests
lint	Lint del código
migrate	Migraciones DB

HERRAMIENTAS CONECTADAS (MCP)

4 activas

● filesytem	Conectado
● github	Conectado
● notion_docs	Conectado
● slack	Conectado

PERMISOS ACTUALES

perfil: seguro

lectura de archivos	: ✅ permitido
escritura de archivos	: ⚠️ preguntar
ejecutar comandos	: ⚠️ preguntar
acceso a red	: ❌ bloqueado
borrar datos	: ❌ bloqueado

CAMBIO PROPUESTO (DIFF)

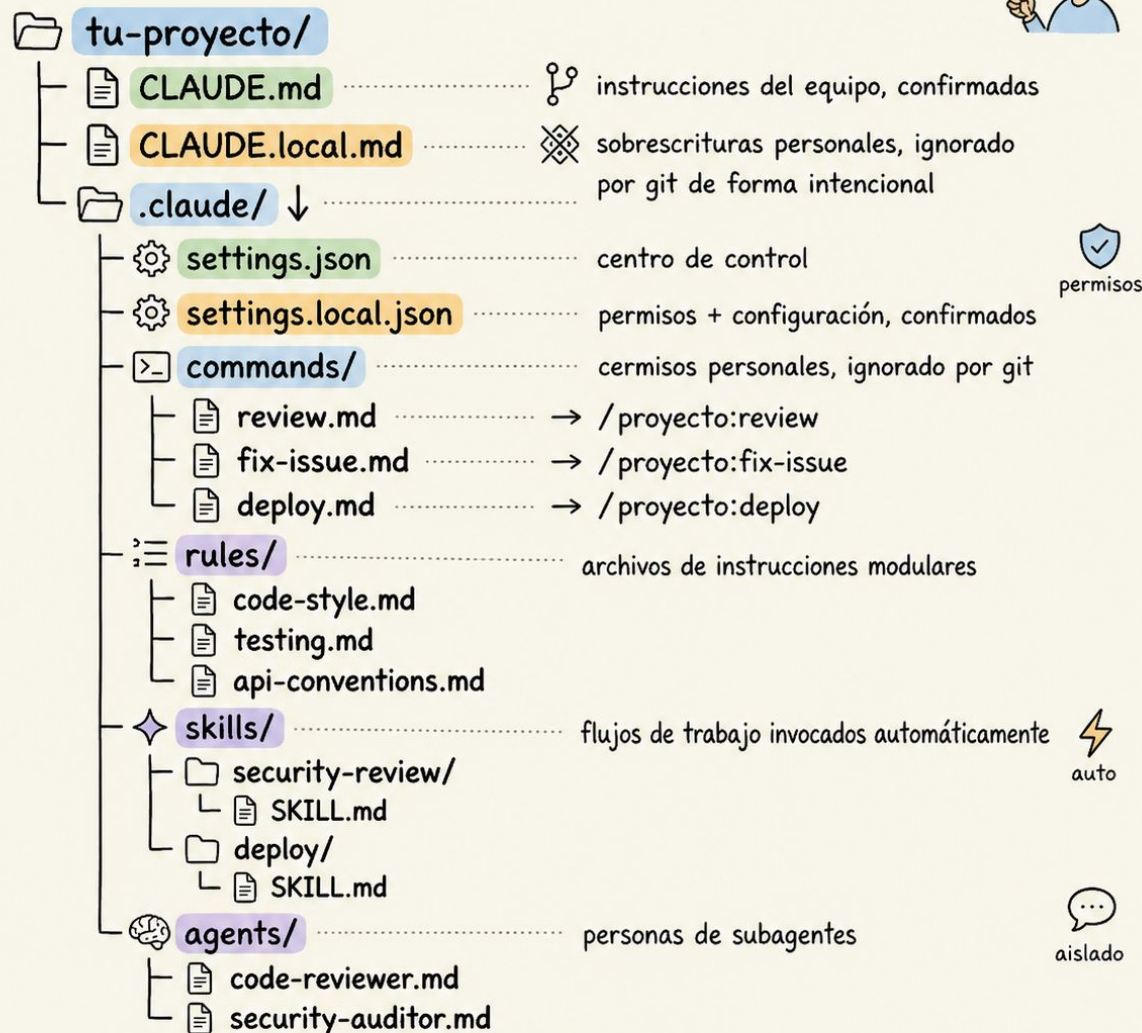
backend/api/views.py

```
45 def get_user(request, user_id):
46 -     user = User.objects.get(id=user_id)
47 +     user = User.objects.filter(id=user_id).first()
48 +     if not user:
49 +         raise Http404("Usuario no encontrado")
```





Anatomía de la carpeta .claude/



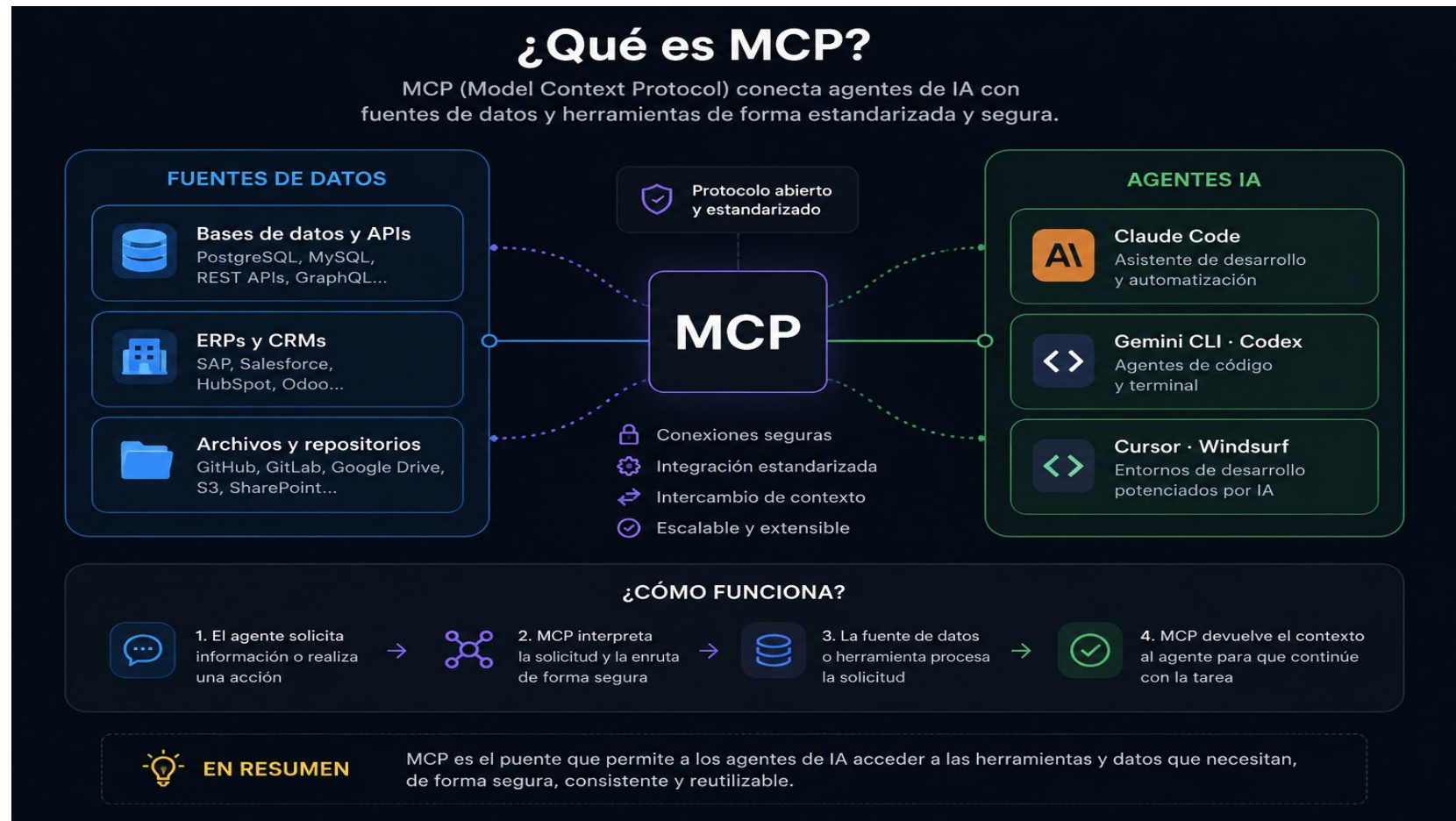
Todo lo que Claude necesita saber sobre tu proyecto está aquí.



MCP: Model Context Protocol



Un estándar abierto para conectar cualquier agente con cualquier sistema externo.



Un servidor puede servir a muchos clientes. Un cliente puede conectarse a muchos servidores.
Escribe una vez, úsalo en todas partes.

MCP Server

Declara capacidades vía un schema JSON:

Herramientas

Funciones que el agente puede llamar (query_db, search_docs, create_ticket)

Resources

Datos que el agente puede leer (archivos, filas de BD, respuestas de API)

Prompts

Plantillas de prompt reusables para tareas comunes

Transport: stdio o HTTP+SSE

Protocolo

Mensajes JSON-RPC 2.0:

1. initialize

Cliente → Servidor: handshake, negociar capacidades

2. tools/list

El cliente descubre available tools

3. tools/call

El cliente invoca un tool, el servidor devuelve resultado

4. resources/read

El cliente lee un resource por URI

Sesión con estado, bidireccional.

MCP Clignt

El agente IA (Claude Code, Cursor, etc.):

Descubre tools al arrancar vía tools/list

Decide qué tool llamar según la intención del usuario

Envía tools/call con argumentos

Recibe resultados estructurados e incorpora al contexto

Config:

claude_desktop_config.json o
.cursor/mcp.json

Spec-Driven Development





Primero definimos qué debe hacer el sistema; después el agente implementa el código.

github.com/github/spec-kit

- Una spec es una especificación clara del comportamiento esperado.
- Define el qué, el por qué, las reglas y los criterios de aceptación.
- El agente implementa el cómo siguiendo la spec como contrato.
- La spec se convierte en tests, ejemplos y casos límite.
- Resultado: menos ambigüedad, más trazabilidad y código más predecible.

Dev tradicional



Dev con agentes IA (Spec-Driven)



01

Constitución

Principios rectores: calidad, testing,
UX, performance

02

Especificar

Describe lo que construyes: user
stories y requisitos

03

Plan

Arquitectura, stack, frameworks,
estrategia

04

Clarificar

Resuelve ambigüedades antes de
codificar

05

Tareas

Descompón el plan en unidades de
trabajo medibles

06

Implementar

Ejecuta con tu agente IA, guiado por
la spec



spec-kit

Toolkit open-source para flujos spec-driven

Repositorio

github.com/github/spec-kit

Quality Gates

```
/speckit.analyze  
/speckit.checklist
```

Agentes Soportados

Claude Code, Copilot, Cursor, Gemini,
Windsurf

Instalación Rápida

```
uv tool install  
specify-cli
```

La pila de la especificación

4 archivos markdown que dirigen todo el flujo

1

CONSTITUTION.md

Principios rectores y valores del proyecto

2

SPECIFICATION.md

Requisitos y user stories

3

IMPL_PLAN.md

Arquitectura y stack tecnológico

4

TASKS.md

Desglose de trabajo y tracking

Ejercicios Opcionales



A RETO A — RAG: Asistente de conocimiento operativo

Dolor

Un equipo tiene documentación dispersa: PDFs, FAQs, procedimientos, notas internas y casos históricos. Buscar respuestas manualmente es lento y propenso a errores.

Stack

Python + Streamlit + LangChain opcional + embeddings + vector store local.

Puede ser Chroma, FAISS o similar. Documentos de ejemplo de cualquier ámbito.

Objetivo

Crear un asistente que responda preguntas usando documentos cargados por el alumno y que cite de dónde sale la respuesta.

- ← Carga de documentos o textos
- ← Indexación semántica
- ← Pregunta en lenguaje natural
- ← Respuesta con fuentes/citas
- ← Aviso cuando no tenga suficiente contexto

B RETO B — Streamlit: Dashboard de datos con insights

Dolor

Muchos análisis de datos acaban en tablas o notebooks que un usuario no técnico no sabe interpretar.

Stack

Python + Streamlit + Pandas + datos sintéticos o CSV existente. Opcional: usar la API de OpenAI para crear resumen ejecutivo

Objetivo

Crear una mini-app interactiva que permita cargar o generar un dataset, visualizar métricas clave y obtener conclusiones claras para un usuario no técnico.

- ← Carga de CSV o generación de datos sintéticos
- ← Limpieza básica y vista previa del dataset
- ← KPIs principales y gráficos simples
- ← Filtros interactivos
- ← Opcional: resumen ejecutivo con API de OpenAI

 ¿Qué enfoque seguiste para resolver el reto?

 ¿Qué funcionó bien y qué podrías haber hecho diferente?

 ¿Qué aprendiste sobre trabajar con IA?





Papers base

- ReAct - razonamiento + acción: arxiv.org/abs/2210.03629
- Reflexion - memoria verbal y auto-mejora: arxiv.org/abs/2303.11366
- Voyager - skills reutilizables: arxiv.org/abs/2305.16291
- SWE-agent - agentes de programación: arxiv.org/abs/2405.15793



Tooling 2026

- MCP Spec + Registry: modelcontextprotocol.io · registry.modelcontextprotocol.io
- OpenAI Agents SDK / AgentKit: platform.openai.com/docs/guides/agents
- LangGraph: docs.langchain.com/oss/python/langgraph
- Microsoft Agent Framework: learn.microsoft.com/en-us/agent-framework/



Formación

- DeepLearning.AI Agentic AI: deeplearning.ai/courses/agentic-ai/



KAHOOT!

Kahoot!





EBIS Business
Techschool

www.ebiseducation.com

